Phase 1: Conception

Project: Build a Data Mart in SQL

BSC in Computer Science

Instructor: Sharam Dadashnia

Topic: ERM and Data Model for Airbnb-style Accommodation Platform

Date: 28th June, 2025

By

Zubair Khan

Matriculation: 92127983

Table of Contents

1. Introduction

The development of the data-driven solution to a digital accommodation platform like Airbnb requires a well-designed and solid database framework. The Entity Relationship Model (ERM) is at the centre of this framework and makes sure that different data points, such as listings and bookings, user reviews, and administrative functions, are logically and efficiently interconnected. This project aims at modelling a system that would support complex interactions and be scalable, data and usable. Using the experience of success and problems of the own platform architecture of Airbnb, this ERM incorporates entity interaction in the form of reservations, payments, multilingual communication, and complaint management.

The scholarly literature highlights the disruptive nature of Airbnb that changed the conventional hospitality paradigm by offering an informal, peer-to-peer service (Guttentag, 2015). The proper ER model makes the system more transparent and manageable, which can help to avoid the risk of failure to enforce it or bypassing the regulatory framework (Leshinsky & Schatz, 2018). The data-driven approach is applied in this design to support user preferences, property regulations, multilingual access, and booking management, therefore, making the platform competitive and operationally viable.

2. Roles

The ERM has a number of user and system roles that have their own responsibilities in the platform. These are **Guests, Hosts, Administrators, Support Staff, and Employees.** The different roles will be assigned to different entities and relationships to encourage workflows specific to roles.

- End-users are **guests** who search and book listings and leave the reviews. They deal mostly with booking, payment, and review information.
- **Hosts** are people who offer accommodation, handle listing, price, availability, and communication with guests.
- **Administrators** deal with user management, platform configuration and conflict resolution.
- The **Support Staff** assists users in troubleshooting and handling requests and provides first-line support.
- **Employees** are the organizational personnel that keep the platforms running, including the hiring process and internal HR.

These functions are consistent with the results of Gurran (2018), who observed that the process of platform governance and user management is becoming increasingly complicated in peer-to-peer hospitality settings. The addition of support functions also indicates the move to hybrid service models with inbuilt professional support at Airbnb (Cheng & Mackenzie, 2022)..

3. Actions

Each role in the system executes defined actions that shape the interaction flow and functional coverage of the platform:

Guest Actions:

- Browse listings using filter parameters (e.g., price, amenities, availability).
- Make reservations and manage check-in/check-out dates.
- Review past stays, providing qualitative and quantitative feedback.
- Update profile preferences and manage payment history.

Host Actions:

- Create and manage listings by uploading photos, setting prices, and defining house rules.
- Confirm or decline booking requests based on calendar availability.
- Monitor income via autogenerated reports.

Administrator Actions:

- Manage user registration and access credentials.
- Review platform activity logs and configure system-wide settings.
- Manage formal complaints and resolve user conflicts.

Support Staff Actions:

- Address user queries via integrated notification systems.
- Support payment or booking error resolution.
- Escalate unresolved complaints to administrators.

Each of these actions is mapped in the ERM with appropriate foreign keys and relationship cardinalities, ensuring a relationally sound database. As Guttentag (2015) and Prayag et al. (2022) emphasize, clear workflows are essential for maintaining platform trust and regulatory compliance in sharing economy models.

4. Data and Functions

The ERM diagram of the Airbnb-style platform has a properly organized data architecture that supports a wide variety of interactions between users and the system-level operations. The data model focuses on the most important entities: User, Accommodation, Booking, Payment, Review,

and Place, and each of them is related to the particular functional requirements that are necessary to have a dynamic hospitality platform. These functions indicate the requirements of real-world application, including support of multiple languages, secure booking and payment processes, and administrative control.

As a guest, the platform allows a data-informed search and reservation of the listings. The Place and Accommodation objects contain the features of Pricing, Availability, and Photos, which enable the user to filter and choose properties that suit them. Customers start the reservation process with the help of the Booking entity and connect with the Payment and Reservation entities to make payments. There are check-in/out processing, recording of transactions and viewing of booking history. The Review table, which is related to the ReviewerID and ReviewedID of the User table, enables the guests to give feedback after staying, which increases the trust in the community and accountability of the platform.

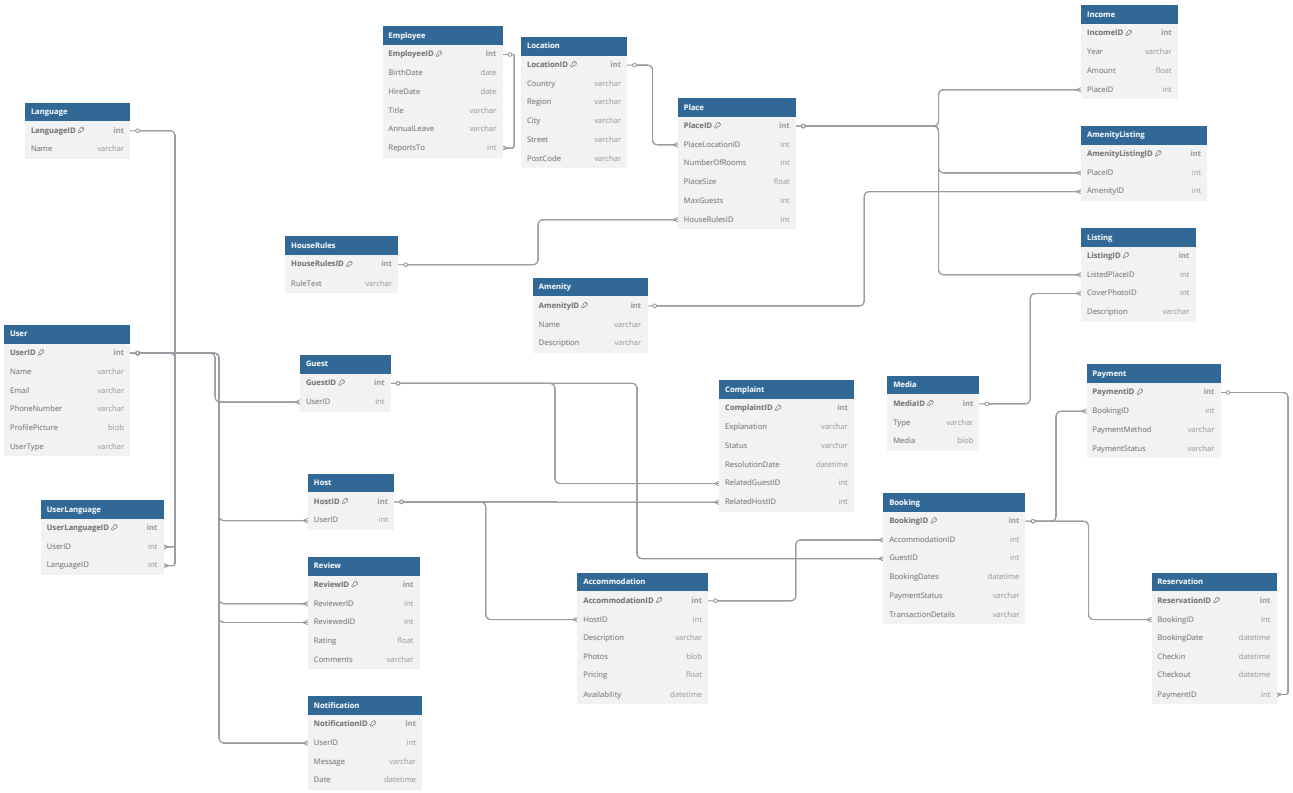
Another significant user group is hosts, who work with such functions as listing management, visualizing of earnings, and communicating with guests. The Listing, Place and AmenityListing entities give the host the capability of specifying the property attributes, related amenities and relevant HouseRules. Also, the Income table provides hosts with information about revenues in the long-term to assess the business performance. The mechanisms to resolve complaints are also implemented; the Complaint table links guests and hosts to dispute data and resolution tracking, increasing the platform transparency and user support.

The administrative and support staff functions of the platform are supported by the higher-level system-level entities like Employee, Notification and UserLanguage. The multilingual feature is done in a many-to-many relationship using the UserLanguage junction table and thus the platform can support internationalization. Event-driven communication is automated through notifications that are associated with UserID and assist in system alerts, reservation confirmations, and policies. The hierarchies of employees can be modeled through a self-referencing ReportsTo relationship, which allows role-based authorizations and delegation of tasks.

At the system level, data integrity and modularity is maintained by foreign key relationship between entities, Booking -> Accommodation, Payment -> Booking, and Reservation -> Payment. The management of the media is also centralized through the Media entity, which is being referenced by Listing through CoverPhotoID, so the media assets can be stored and retrieved efficiently. Such an extensive relational arrangement is consistent with academic research on the strong platform data design, which places a particular focus on the significance of normalized schemas regarding security, performance, and scalability (Ozdemir & Turker, 2019).

In this way, the data and the related functions do not only fulfill the expectations of the users in terms of their roles but also support platform governance, communication, and extensibility at the backend.

5. ERM (Entity Relationship Model):



Triple & Recursive Relationships:

The triple and recursive relations in the given ERM diagram are the key patterns of interactions in a solid accommodation platform.

The triple relationships are seen when there are three entities which are related to each other in a mutual context. As an example, in the BookingGuestProperty structure, there is an obvious triangular relationship between a Guest, the Booking they make and the Property (via Accommodation and Place) they book. This is a multi-entity dependency in the booking flow. The other triple structure can be seen in the ReviewBookingGuest relationship, where a Guest writes a Review based on a Booking, not only capturing feedback, but also putting it in context of the guest experience and booking event. All these interdependencies guarantee data consistency and deep analytics of user behavior and property performance.

On the scale of recursive relationships, the ERM discloses such intricacy in entities such as User, especially in support interactions. A User (Host or Guest) can engage with support in a cyclic way, by opening several support tickets and getting feedback or actions by support staff, which are also users. This User User Support Ticket User logic presents a self-referencing loop, and that is a feature of recursive relationships in which the entity is related to itself in more than one capacity. Likewise, recursive relationships can be found in the Employee table in the field called ReportsTo, indicating

that an employee (which can be a support or management entity) can report to an employee, creating a loop in the organization hierarchy.

The triple and recursive relationship mappings are sophisticated representations of relationships, which are carefully designed in order to represent subtle real-life interactions and decision paths, which increases the functional fidelity and analytical strength of the system.

Data Dictionary:

User Table

Attribute Name	**Data Type**	**Description**
UserID (PK)	int	A unique identifier assigned to each user.
Name	varchar	Full name of the user.
Email	varchar	Email address used for user communication.
PhoneNumber	varchar	Contact number of the user.
ProfilePicture	blob	Binary data representing user's profile photo.
UserType	varchar	Specifies whether the user is a host or guest.

Guest Table

Attribute Name	**Data Type**	**Description**
GuestID (PK)	int	Primary key uniquely identifying a guest.
UserID (FK)	int	Foreign key referring to the user table.

Host Table

Attribute Name	**Data Type**	**Description**
HostID (PK)	int	Unique identifier for a host profile.

Attribute Name	Data Type	Description
UserID (FK)	int	Links host to a specific user entry.

Employee Table

Attribute Name	Data Type	Description
EmployeeID (PK)	int	Unique key for an employee.
BirthDate	date	Date of birth of the employee.
HireDate	date	Date the employee officially joined.
Title	varchar	Designation or job title of the employee.
AnnualLeave	varchar	Information on allotted vacation days.
ReportsTo (FK)	int	Supervisor or manager reference in employee table.

Language Table

Attribute Name	Data Type	Description
LanguageID (PK)	int	Unique ID for each language record.
Name	varchar	Language name (e.g., English, Spanish).

UserLanguage

Attribute	Description	Data Type
UserLanguageID	Primary key to uniquely identify user-language relation.	int
UserID	Foreign key linking to a user entry.	int
LanguageID	Foreign key linking to a language entry.	int

Review

Attribute	Description	Data Type
ReviewID	Unique identifier for the review entry.	int
ReviewerID	References the user who posted the review.	int
ReviewedID	Refers to the user being reviewed.	int
Rating	Numeric score assigned in the review.	float
Comments	Written feedback or opinion included in the review.	varchar

Notification

Attribute	Description	Data Type
NotificationID	Identifier to distinguish each notification.	int
UserID	Foreign key connecting to the user who receives the notification.	int
Message	Text content delivered in the notification.	varchar
Date	Timestamp for when the notification was issued.	datetime

Accommodation

Attribute	Description	Data Type
AccommodationID	Unique identifier for the accommodation record.	int
HostID	Foreign key pointing to the host offering this accommodation.	int
Description	Brief overview of the accommodation features.	varchar
Photos	Image data or file representing accommodation visuals.	blob
Pricing	The cost per booking or night.	float
Availability	Dates and times the accommodation is available.	datetime

Booking

Attribute	Description	Data Type
BookingID	Primary key for each booking transaction.	int
AccommodationID	Foreign key linking to the booked accommodation.	int
GuestID	Foreign key referencing the user who made the booking.	int
BookingDates	Period of reservation including check-in and check-out.	datetime
PaymentStatus	Describes whether the booking payment is complete, pending, or failed.	varchar
TransactionDetails	Additional notes or data related to the transaction.	varchar

Payment

Attribute	Description	Data Type
PaymentID	Primary key uniquely identifying a payment.	int
BookingID	References the associated booking record.	int
PaymentMethod	Describes the method used for payment.	varchar
PaymentStatus	Current status of the payment transaction.	varchar

Reservation

Attribute	Description	Data Type
ReservationID	Unique identifier for each reservation.	int
BookingID	Foreign key pointing to the related booking.	int
BookingDate	Date the reservation was made.	datetime
Checkin	Scheduled check-in date and time.	datetime
Checkout	Scheduled check-out date and time.	datetime

Attribute	Description	Data Type
PaymentID	References the related payment record.	int

Complaint

Attribute	Description	Data Type
ComplaintID	Unique ID for each complaint entry.	int
Explanation	Details provided about the complaint.	varchar
Status	Current resolution status of the complaint.	varchar
ResolutionDate	Date when the complaint was addressed.	datetime
RelatedGuestID	Guest ID involved in the complaint.	int
RelatedHostID	Host ID involved in the complaint.	int

Location

Attribute	Description	Data Type
LocationID	Primary key for each location.	int
Country	Country of the location.	varchar
Region	State or regional subdivision.	varchar
City	City name.	varchar
Street	Street address.	varchar
PostCode	Postal or ZIP code.	varchar

Place

Attribute	Description	Data Type
PlaceID	Unique identifier for a place entity.	int
PlaceLocationID	Foreign key to the location of the place.	int
NumberOfRooms	Number of rooms available in the place.	int
PlaceSize	Measurement of the place's area.	float
MaxGuests	Maximum number of guests allowed.	int
HouseRulesID	Foreign key linking to applicable house rules.	int

Listing

Field Name	Data Type	Description
ListingID	int	Primary key, uniquely identifies a listing
ListedPlaceID	int	Foreign key referencing the listed place
CoverPhotoID	int	Foreign key referencing media used
Description	varchar	Textual details about the listing

Media

Field Name	Data Type	Description
MediaID	int	Primary key for media assets
Type	varchar	Type of media (image, video, etc.)
Media	blob	Binary large object storing the file

HouseRules

Field Name	Data Type	Description
HouseRulesID	int	Primary key, uniquely identifies rules
RuleText	varchar	Text of the house rule

Income

Field Name	Data Type	Description
IncomeID	int	Primary key, uniquely identifies income
Year	varchar	Year in which income was recorded
Amount	float	Total income amount
PlaceID	int	Foreign key referencing related property

Amenity

Field Name	Data Type	Description
AmenityID	int	Primary key for amenities
Name	varchar	Name or title of the amenity
Description	varchar	Detailed description of the amenity

AmenityListing

Field Name	Data Type	Description
AmenityListingID	int	Primary key for amenity-place association
PlaceID	int	Foreign key referencing the place entity
AmenityID	int	Foreign key referencing the amenity entity

5. Summary

The thorough ERM (Entity Relationship Model) created of Airbnb is a carefully thought-out and organized data model that serves the complex nature of a massive online accommodation service. The design includes a broad scope of entities and each entity is a representation of the main components of operation, including users (guests and hosts), employees, locations, accommodations, reservations, and transactions. Such architecture allows accurate data flow capture and management throughout the system. The interconnectedness between the entities, i.e., the connections between bookings and payments, guests and reviews, hosts and accommodations, and places and amenities, demonstrates the extent of normalization and integrity imposed on the model.

In addition, the use of associative tables such as UserLanguage and AmenityListing also makes many-to-many relationships to be managed effectively, enhancing flexibility and scalability. User interaction and transparency of operations is facilitated by logical extensions of Review, Complaint and Notification modules. The application of the Chen notation assisted in establishing cardinalities and recursion in the required areas particularly in situations that involved Employee hierarchy or Complaint associations. On the whole, this ERM does not only facilitate the scope of the Airbnb operations, but it also guarantees flexibility to the future changes, consistency and clarity of the data within a complex transactional environment.

6. References

Guttentag, D. (2015). Airbnb: Disruptive innovation and the rise of an informal tourism accommodation sector. *Tourism Management, 52*, 278–289. https://doi.org/10.1080/13683500.2013.827159

Gurran, N. (2018). Global home-sharing, local communities and the Airbnb debate: A planning research agenda. *Taylor & Francis*.

Leshinsky, R., & Schatz, L. (2018). “I don't think my landlord will find out:” Airbnb and the challenges of enforcement. *Taylor & Francis*.

Wagner, G., & Prester, J. (2019). Information systems research on digital platforms for knowledge work: A scoping review. *CORE*. Full text PDF

Ozdemir, G., & Turker, D. (2019). Institutionalization of the sharing in the context of Airbnb: A systematic literature review and content analysis. *International Journal of Economic and Administrative Studies, 1*(1), 1–20. https://doi.org/10.1080/13032917.2019.1669686

Trabucchi, D., Moretto, A., & Buganza, T. (2020). Disrupting the disruptors or enhancing them? How blockchain reshapes two-sided platforms. *Journal of Product Innovation Management*. https://doi.org/10.1111/jpim.12557. Full text PDF

Wang, Y., Asaad, Y., & Filieri, R. (2020). What makes hosts trust Airbnb? Antecedents of hosts' trust toward Airbnb and its impact on continuance intention. *Journal of Travel Research*. https://doi.org/10.1177/0047287519855135. Full text PDF

Stivaktakis, I. (2021). *A multi-dimensional data analytics model for quality assessment in the hospitality, leisure and tourism sector* [Master's thesis, University of Nicosia]. University of Nicosia. Full text PDF

Cheng, M., & Mackenzie, S. H. (2022). Airbnb impacts on host communities in a tourism destination: An exploratory study of stakeholder perspectives in Queenstown, New Zealand. *Taylor & Francis*.

Prayag, G., Ozanne, L. K., & Martin-Neuninger, R. (2022). Integrating MLP and 'after ANT' to understand perceptions and responses of regime actors to Airbnb. *Taylor & Francis*.

Trușculescu, A. E., & Drăghici, A. (2023). *Business valuation across the industry life cycle: Focus on internet-enabled businesses* [Doctoral dissertation, Politehnica University of Timisoara]. Full text PDF

Adere, E. M. (2024). Toward a definition of blockchain: Analyzing definitions to propose a definition. *Information - Wissenschaft & Praxis*. Full text PDF

Paliszkiewicz, J., & Chen, K. (2024). *Trust in social and business relations*. Routledge. Landing page

Liu, B., Moyle, B., Kralj, A., & Vada, S. (2025). Rethinking travel companionship: An alternative conceptual model and future research agenda. *Journal of Hospitality & Tourism Research*. https://doi.org/10.1177/10963480241258084. Full text PDF

Sanli, M. (2025). *Digital transformations in the hotel industry* [Doctoral dissertation, Long Island University]. ProQuest Dissertations Publishing. Full text PDF

Lukyanenko, R. (2408). The MAGIC of data management: Understanding the value and activities of data management. *arXiv*. https://doi.org/10.48550/arXiv.2408.07607. Full text PDF

Task:
A Use-Case of booking a hotel room: Airbnb

Phase 2: Development

BSC in Computer Science

Instructor: Sharam Dadashnia

Date: , 2026

Submitted By: Zubair Khan

Matriculation No.: 92127983

Github URL: https://github.com/zbr-khn/SQL/tree/main

Introduction

This project implements a relational database management system for an Airbnb booking platform using MySQL. The database was designed from the Entity Relationship Diagram developed in Phase 1 and includes 21 related tables representing users, hosts, guests, bookings, reservations, payments, listings and supporting entities.

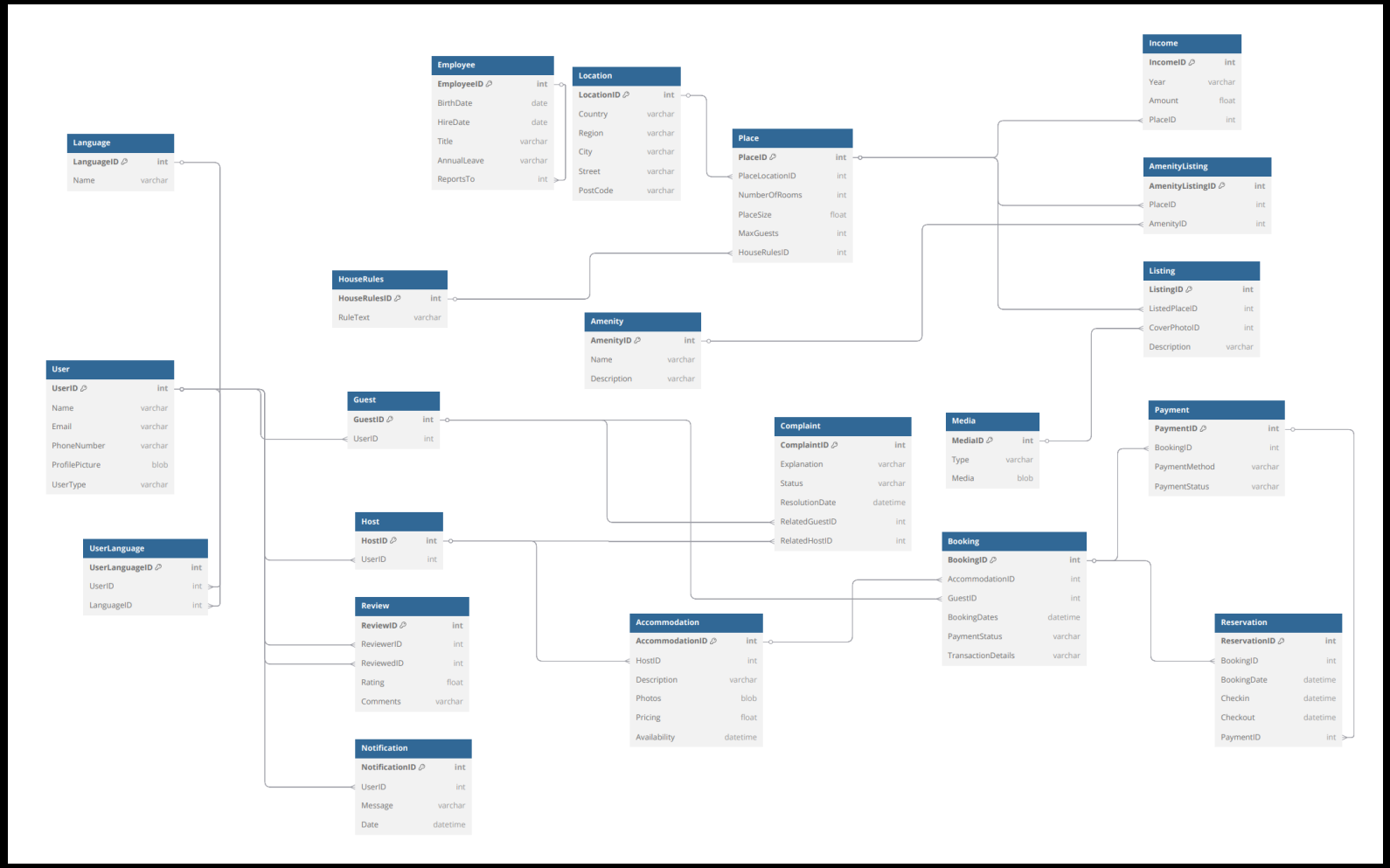
Technical highlights include:

- The database management system was developed using MySQL and is based on the Entity Relationship (ER) Model designed in Phase 1. The database consists of well-structured tables that are connected through primary and foreign key relationships, ensuring data integrity and consistency throughout the system. This relational structure allows different entities to interact efficiently while minimizing data redundancy.
- The system supports the core functionality of an Airbnb-style booking platform, including user management, property listings, bookings, reservations, payments, reviews, and notifications. Each feature is stored in its own table while remaining connected through relationships, making the database easy to manage, maintain, and expand in the future.
- By following the principles of database normalization and relational database design, the system provides accurate data storage, reliable relationships between tables, and efficient data retrieval through SQL queries. This structure ensures that the platform can handle everyday booking operations while maintaining consistency, scalability, and overall performance.

ER Diagram

At the centre lies an Entity Relationship Model - this structure guides how pieces within the system link together. Built around key elements such as Users, Guests, Hosts, and Accommodations, it manages essential functions including reservations and financial transactions.

Reality checks shape how users trust a system, so features such as guest feedback, service issues, and facility details were added into the design. Because accuracy matters when records interact, related entries connect through enforced links across tables - this means each reservation or financial record aligns only with its matching person and rental space.



User Tables

Among stored records, the User table holds core details like names, emails, and phone numbers for every individual using the system. Built into the structure of the database, this main reference point supports two distinct roles - Guest and Host - branching out from shared attributes. Through this setup, duplication drops while consistency across entries rises.
Every user gets a unique spot through UserID, acting as the main identifier. What makes one person different from another shows up clearly when UserType sorts roles apart naturally.
By cutting repeated entries, this setup keeps information more uniform while making connections to tables like Guest, Host, or Notification easier to handle. Though cleaner structure emerges, managing links becomes less tangled over time through subtle improvements in layout flow.
Foundational to the relational setup, this table links to several related entities via external key ties.

SQL Statement:

```
CREATE TABLE `user` (
  `UserID` int NOT NULL,
  `Name` varchar(100) DEFAULT NULL,
  `Email` varchar(100) DEFAULT NULL,
  `PhoneNumber` varchar(15) DEFAULT NULL,
  `ProfilePicture` blob,
  `UserType` varchar(20) DEFAULT NULL,
  PRIMARY KEY (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM User;

Result:

UserID	Name	Email	PhoneNumber	ProfilePicture	UserType
1	Michael Keith	michael@gmail.com	9999999999	NULL	Guest
2	Ali Ahmed	ali@gmail.com	8888888888	NULL	Host
3	Sara Khan	sara@gmail.com	7777777777	NULL	Guest
4	John Doe	john@gmail.com	6666666666	NULL	Host
5	User5	user5@mail.com	9000000005	NULL	Guest
6	User6	user6@mail.com	9000000006	NULL	Host
7	User7	user7@mail.com	9000000007	NULL	Guest
8	User8	user8@mail.com	9000000008	NULL	Host
9	User9	user9@mail.com	9000000009	NULL	Guest
10	User10	user10@mail.com	9000000010	NULL	Host
11	User11	user11@mail.com	9000000011	NULL	Guest
12	User12	user12@mail.com	9000000012	NULL	Host
13	User13	user13@mail.com	9000000013	NULL	Guest
14	User14	user14@mail.com	9000000014	NULL	Host
15	User15	user15@mail.com	9000000015	NULL	Guest
16	User16	user16@mail.com	9000000016	NULL	Host
17	User17	user17@mail.com	9000000017	NULL	Guest
18	User18	user18@mail.com	9000000018	NULL	Host
19	User19	user19@mail.com	9000000019	NULL	Guest
20	User20	user20@mail.com	9000000020	NULL	Host
21	User21	user21@mail.com	9000000021	NULL	Guest
22	User22	user22@mail.com	9000000022	NULL	Host
23	User23	user23@mail.com	9000000023	NULL	Guest
24	User24	user24@mail.com	9000000024	NULL	Host
NULL	NULL	NULL	NULL	NULL	NULL

Guest Table

Among those using the platform to reserve stays, some appear in the Guest table. This setup comes from the User entity, linked by UserID acting as a reference point across tables.
Because a foreign key is applied, every guest entry links back to a valid user already present. That connection stops stray data from appearing by accident. Consistency throughout the database holds firm when relationships stay anchored like this. Without such structure, mismatches could grow unchecked.
Splitting Guest from User lets the system organize data by roles yet keep structure clean. Because of this separation, tasks like managing reservations or feedback run smoothly for guests alone - without repeating core details meant for all users.
With its structure designed around user interactions, this table supports functions tied to reservations while connecting individuals to records like Booking and Review. Though unseen, it forms pathways that allow data exchange across key operational points in the system.

SQL Statement:

```
CREATE TABLE `guest` (
  `GuestID` int NOT NULL,
  `UserID` int DEFAULT NULL,
  PRIMARY KEY (`GuestID`),
  KEY `UserID` (`UserID`),
  CONSTRAINT `guest_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Guest;

Result:

GuestID	UserID
1	1
2	3
3	5
13	6
4	7
14	8
5	9
15	10
6	11
16	12
7	13
17	14
8	15
18	16
9	17
19	18
10	19
20	20
11	21
21	22
12	23
22	24
NULL	NULL

Host Table

The Host table represents users who provide accommodations on the platform. Similar to the Guest table, it is derived from the User entity and maintains a relationship through the foreign key (UserID).
Because of the foreign key, each host links only to an existing user within the system. Through this constraint, accuracy across related data elements remains intact. Without such alignment, mismatches could appear unexpectedly. Ensuring these connections exist reduces chances for errors over time.
One reason the structure splits Host from User lies in enabling roles without disrupting normalized design. Where management of stays and rentals falls to hosts, a distinct table becomes necessary. This connection ties individual profiles to places and listings through structured relationships.
Ownership records rely on this structure to link hosts with their listed properties. Where one entry stands, a connection forms between provider and place. Though simple in design, its role supports accurate tracking across the system. From here, assignments gain clarity without extra steps. Behind each listing sits this foundation, silent but required.

SQL Statement:

```
CREATE TABLE `host` (
  `HostID` int NOT NULL,
  `UserID` int DEFAULT NULL,
  PRIMARY KEY (`HostID`),
  KEY `UserID` (`UserID`),
  CONSTRAINT `host_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Host;
```

Result:

HostID	UserID
15	1
1	2
13	2
16	3
2	4
14	4
17	5
3	6
18	7
4	8
19	9
5	10
20	11
6	12
21	13
7	14
22	15
8	16
9	18
10	20
11	22
12	24
NULL	NULL

Accommodation Table

One entry in the Accommodation table stands for a property offered by a host via the system. Found within this structure, every lodging links to one provider using HostID as a reference point. Connection forms when property details meet ownership records across datasets.
Essential details appear within this structure: nightly cost, availability status, descriptive notes, along with data tied to location. Preferences guide user choices during browsing tasks involving lodging options. Information flows in a way that supports selection processes without extra steps.
Because foreign keys are applied, data connections stay accurate, tying each accommodation to an existing host along with associated items like Place or Listing. With this structure in place, lookups proceed smoothly while uniformity throughout the database remains intact.
Separate storage of lodging details enables normalized design, yet still supports growth when handling many entries along with their properties. Though organized apart, the arrangement adapts smoothly as listing volumes increase together with related features.
At the center stands this table, holding essential details for property entries. From it, booking functions emerge alongside guest reviews and reservation handling. Its structure supports key processes across the platform. Without deviation, each operation ties back to its framework.

SQL Statement:

```
CREATE TABLE `accommodation` (
  `AccommodationID` int NOT NULL,
  `HostID` int DEFAULT NULL,
  `Title` varchar(100) DEFAULT NULL,
  `Location` varchar(100) DEFAULT NULL,
  `PricePerNight` decimal(10,2) DEFAULT NULL,
  `Description` varchar(255) DEFAULT NULL,
  `Photos` blob,
  `Availability` datetime DEFAULT NULL,
  `PlaceID` int DEFAULT NULL,
  PRIMARY KEY (`AccommodationID`),
  KEY `HostID` (`HostID`),
  KEY `PlaceID` (`PlaceID`),
  CONSTRAINT `accommodation_ibfk_1` FOREIGN KEY (`HostID`) REFERENCES `host` (`HostID`),
  CONSTRAINT `accommodation_ibfk_2` FOREIGN KEY (`PlaceID`) REFERENCES `place` (`PlaceID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

AccommodationID	HostID	Title	Location	PricePerNight	Description	Photos	Availability	PlaceID
1	1	Sea View Apartment	Goa	5000.00	NULL	NULL	NULL	NULL
2	2	Mountain Cabin	Manali	3000.00	NULL	NULL	NULL	NULL
3	1	Stay 1	Delhi	2000.00	Nice place 1	NULL	2024-05-01 00:00:00	1
4	2	Stay 2	Delhi	2500.00	Nice place 2	NULL	2024-05-02 00:00:00	2
5	3	Stay 3	Delhi	3000.00	Nice place 3	NULL	2024-05-03 00:00:00	3
6	4	Stay 4	Delhi	3500.00	Nice place 4	NULL	2024-05-04 00:00:00	4
7	5	Stay 5	Delhi	4000.00	Nice place 5	NULL	2024-05-05 00:00:00	5
8	6	Stay 6	Bangalore	4500.00	Nice place 6	NULL	2024-05-06 00:00:00	6
9	7	Stay 7	Bangalore	5000.00	Nice place 7	NULL	2024-05-07 00:00:00	7
10	8	Stay 8	Bangalore	5500.00	Nice place 8	NULL	2024-05-08 00:00:00	8
11	9	Stay 9	Bangalore	6000.00	Nice place 9	NULL	2024-05-09 00:00:00	9
12	10	Stay 10	Bangalore	6500.00	Nice place 10	NULL	2024-05-10 00:00:00	10
13	11	Stay 11	Mumbai	7000.00	Nice place 11	NULL	2024-05-11 00:00:00	11
14	12	Stay 12	Mumbai	7500.00	Nice place 12	NULL	2024-05-12 00:00:00	12
15	13	Stay 13	Mumbai	8000.00	Nice place 13	NULL	2024-05-13 00:00:00	13
16	14	Stay 14	Mumbai	8500.00	Nice place 14	NULL	2024-05-14 00:00:00	14
17	15	Stay 15	Mumbai	9000.00	Nice place 15	NULL	2024-05-15 00:00:00	15
18	16	Stay 16	Delhi	9500.00	Nice place 16	NULL	2024-05-16 00:00:00	16
19	17	Stay 17	Delhi	10000.00	Nice place 17	NULL	2024-05-17 00:00:00	17
20	18	Stay 18	Delhi	10500.00	Nice place 18	NULL	2024-05-18 00:00:00	18
21	19	Stay 19	Delhi	11000.00	Nice place 19	NULL	2024-05-19 00:00:00	19
22	20	Stay 20	Delhi	11500.00	Nice place 20	NULL	2024-05-20 00:00:00	20
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Test Query:

```
SELECT * FROM Accommodation;
```

Booking Table

Reservations appear in the Booking table, tied to guests and their chosen stays. Through GuestID and AccommodationID, links form across data points. Payment condition alongside transaction records offer insight into monetary flow per reservation. Relationships gain structure when identifiers connect entities behind the scenes.

From time to time, this table holds essential data about stays - dates when guests arrive and leave, plus related booking records. What connects visitors to their selected accommodations is found here.

Because foreign keys are applied, data connections remain accurate by associating every reservation with existing guest and lodging entries. As a result, monitoring stays organized while creating space for steps like transaction handling or feedback entry. Though structure appears rigid, it allows fluid movement through related tasks.

This structure functions primarily as a central record for transactions, establishing the base layer of handling reservations across the platform. While essential, it operates quietly beneath higher-level processes, supporting data integrity through consistent entry tracking.

SQL Statement:

```
CREATE TABLE `booking` (
  `BookingID` int NOT NULL,
  `GuestID` int DEFAULT NULL,
  `AccommodationID` int DEFAULT NULL,
  `CheckInDate` date DEFAULT NULL,
  `CheckOutDate` date DEFAULT NULL,
  `TotalPrice` decimal(10,2) DEFAULT NULL,
  `PaymentStatus` varchar(50) DEFAULT NULL,
  `TransactionDetails` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`BookingID`),
  KEY `GuestID` (`GuestID`),
  KEY `AccommodationID` (`AccommodationID`),
  CONSTRAINT `booking_ibfk_1` FOREIGN KEY (`GuestID`) REFERENCES `guest` (`GuestID`),
  CONSTRAINT `booking_ibfk_2` FOREIGN KEY (`AccommodationID`) REFERENCES `accommodation` (`AccommodationID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

BookingID	GuestID	AccommodationID	CheckInDate	CheckOutDate	TotalPrice	PaymentStatus	TransactionDetails
1	1	1	2024-04-10	2024-04-15	25000.00	NULL	NULL
2	2	2	2024-05-01	2024-05-05	12000.00	NULL	NULL
3	1	1	2024-06-01	2024-06-05	8000.00	Paid	TXN001
4	2	2	2024-06-02	2024-06-06	9000.00	Paid	TXN002
5	3	3	2024-06-03	2024-06-07	10000.00	Pending	TXN003
6	4	4	2024-06-04	2024-06-08	11000.00	Paid	TXN004
7	5	5	2024-06-05	2024-06-09	12000.00	Paid	TXN005
8	6	6	2024-06-06	2024-06-10	13000.00	Pending	TXN006
9	7	7	2024-06-07	2024-06-11	14000.00	Paid	TXN007
10	8	8	2024-06-08	2024-06-12	15000.00	Paid	TXN008
11	9	9	2024-06-09	2024-06-13	16000.00	Pending	TXN009
12	10	10	2024-06-10	2024-06-14	17000.00	Paid	TXN010
13	11	11	2024-06-11	2024-06-15	18000.00	Paid	TXN011
14	12	12	2024-06-12	2024-06-16	19000.00	Pending	TXN012
15	13	13	2024-06-13	2024-06-17	20000.00	Paid	TXN013
16	14	14	2024-06-14	2024-06-18	21000.00	Paid	TXN014
17	15	15	2024-06-15	2024-06-19	22000.00	Pending	TXN015
18	16	16	2024-06-16	2024-06-20	23000.00	Paid	TXN016
19	17	17	2024-06-17	2024-06-21	24000.00	Paid	TXN017
20	18	18	2024-06-18	2024-06-22	25000.00	Pending	TXN018
21	19	19	2024-06-19	2024-06-23	26000.00	Paid	TXN019
22	20	20	2024-06-20	2024-06-24	27000.00	Paid	TXN020
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Test Query:

```
SELECT * FROM Booking;
```

Payment Table

The Payment table holds transaction details tied to reservations. Connected via BookingID, it references entries in the Booking table. This link ensures only legitimate reservation records receive associated payments. Each financial entry traces back reliably through this relation.
This table holds information like payment method, along with its current status and the associated date, helping the system monitor money movements reliably. Payment results can be followed over time due to this structure, an aspect critical to confirming reservations accurately while maintaining stable operations.
Because foreign keys are applied, data links stay accurate across bookings and payments. As a result, transaction details remain aligned without drift over time. Financial clarity emerges naturally when record flows follow structured paths. Processing speed improves where relationships are clearly defined ahead of time.

SQL Statement:

```
CREATE TABLE `payment` (
  `PaymentID` int NOT NULL,
  `BookingID` int DEFAULT NULL,
  `PaymentMethod` varchar(50) DEFAULT NULL,
  `PaymentStatus` varchar(50) DEFAULT NULL,
  `PaymentDate` date DEFAULT NULL,
  PRIMARY KEY (`PaymentID`),
  KEY `BookingID` (`BookingID`),
  CONSTRAINT `payment_ibfk_1` FOREIGN KEY (`BookingID`) REFERENCES `booking` (`BookingID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Payment;
```

Result:

PaymentID	BookingID	PaymentMethod	PaymentStatus	PaymentDate
1	1	Credit Card	Paid	2024-06-01
2	2	PayPal	Paid	2024-06-02
3	3	Debit Card	Pending	2024-06-03
4	4	Bank Transfer	Paid	2024-06-04
5	5	Credit Card	Paid	2024-06-05
6	6	PayPal	Pending	2024-06-06
7	7	Debit Card	Paid	2024-06-07
8	8	Credit Card	Paid	2024-06-08
9	9	Bank Transfer	Pending	2024-06-09
10	10	PayPal	Paid	2024-06-10
11	11	Credit Card	Paid	2024-06-11
12	12	Bank Transfer	Pending	2024-06-12
13	13	Debit Card	Paid	2024-06-13
14	14	Credit Card	Paid	2024-06-14
15	15	Bank Transfer	Pending	2024-06-15
16	16	PayPal	Paid	2024-06-16
17	17	Credit Card	Paid	2024-06-17
18	18	Bank Transfer	Pending	2024-06-18
19	19	Debit Card	Paid	2024-06-19
20	20	Credit Card	Paid	2024-06-20
NULL	NULL	NULL	NULL	NULL

Reservation Table

A booking confirmation appears within the Reservation table, which holds supplementary data for managing stays. Connected via BookingID, this entry ties back to the primary record in the Booking table.
This table holds details like the booking date, when guests arrive and leave, also connects payments via PaymentID. With it, each reservation moves clearly from start to finish, while alignment between bookings and transactions stays consistent.
With foreign keys in place, data accuracy improves through enforced links between reservations, bookings, and payments. Because each entry ties back correctly, handling reservation records becomes more systematic.
Efficiency emerges not from added tools but from structured relationships. Organization within the system grows as dependencies remain consistent. Valid connections form the base for reliable operations.
This structure divides reservation monitoring from the process of making bookings, which increases flexibility within the system while enabling later additions like managing changes or cancellations. While separation occurs here, functionality remains clear and distinct across components.

SQL Statement:

```
CREATE TABLE `reservation` (
  `ReservationID` int NOT NULL,
  `BookingID` int DEFAULT NULL,
  `BookingDate` datetime DEFAULT NULL,
  `CheckIn` datetime DEFAULT NULL,
  `CheckOut` datetime DEFAULT NULL,
  `PaymentID` int DEFAULT NULL,
  PRIMARY KEY (`ReservationID`),
  KEY `BookingID` (`BookingID`),
  KEY `PaymentID` (`PaymentID`),
  CONSTRAINT `reservation_ibfk_1` FOREIGN KEY (`BookingID`) REFERENCES `booking` (`BookingID`),
  CONSTRAINT `reservation_ibfk_2` FOREIGN KEY (`PaymentID`) REFERENCES `payment` (`PaymentID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Reservation;
```

Result:

ReservationID	BookingID	BookingDate	CheckIn	CheckOut	PaymentID
1	1	2024-05-25 00:00:00	2024-06-01 00:00:00	2024-06-05 00:00:00	1
2	2	2024-05-26 00:00:00	2024-06-02 00:00:00	2024-06-06 00:00:00	2
3	3	2024-05-27 00:00:00	2024-06-03 00:00:00	2024-06-07 00:00:00	3
4	4	2024-05-28 00:00:00	2024-06-04 00:00:00	2024-06-08 00:00:00	4
5	5	2024-05-29 00:00:00	2024-06-05 00:00:00	2024-06-09 00:00:00	5
6	6	2024-05-30 00:00:00	2024-06-06 00:00:00	2024-06-10 00:00:00	6
7	7	2024-05-31 00:00:00	2024-06-07 00:00:00	2024-06-11 00:00:00	7
8	8	2024-06-01 00:00:00	2024-06-08 00:00:00	2024-06-12 00:00:00	8
9	9	2024-06-02 00:00:00	2024-06-09 00:00:00	2024-06-13 00:00:00	9
10	10	2024-06-03 00:00:00	2024-06-10 00:00:00	2024-06-14 00:00:00	10
11	11	2024-06-04 00:00:00	2024-06-11 00:00:00	2024-06-15 00:00:00	11
12	12	2024-06-05 00:00:00	2024-06-12 00:00:00	2024-06-16 00:00:00	12
13	13	2024-06-06 00:00:00	2024-06-13 00:00:00	2024-06-17 00:00:00	13
14	14	2024-06-07 00:00:00	2024-06-14 00:00:00	2024-06-18 00:00:00	14
15	15	2024-06-08 00:00:00	2024-06-15 00:00:00	2024-06-19 00:00:00	15
16	16	2024-06-09 00:00:00	2024-06-16 00:00:00	2024-06-20 00:00:00	16
17	17	2024-06-10 00:00:00	2024-06-17 00:00:00	2024-06-21 00:00:00	17
18	18	2024-06-11 00:00:00	2024-06-18 00:00:00	2024-06-22 00:00:00	18
19	19	2024-06-12 00:00:00	2024-06-19 00:00:00	2024-06-23 00:00:00	19
20	20	2024-06-13 00:00:00	2024-06-20 00:00:00	2024-06-24 00:00:00	20
NULL	NULL	NULL	NULL	NULL	NULL

Review Table

Found within the system, the Review table holds evaluations made by individuals about stays and exchanges between people. Connections form among participants via designated roles - one offering judgment, the other receiving it - linked through reference identifiers from user records.
This structure holds details like feedback scores alongside written reviews, which helps the platform monitor consistency while following how users respond over time.
Trust grows when guests share experiences after stays. Hosts gain clarity through guest reflections over time. Quality tends to rise where feedback becomes visible. Reputation shifts subtly with each written comment. The system holds steady mainly because voices accumulate.
Because foreign keys are applied, data connections remain accurate by assigning each review to an existing user. As a result, past interactions become reliable references for those evaluating options later.
This structure supports reliability on the system through clear visibility along with changes shaped by user input. Trust grows where actions are visible, adjustments responsive.

SQL Statement:

```
CREATE TABLE `review` (
  `ReviewID` int NOT NULL,
  `ReviewerID` int DEFAULT NULL,
  `ReviewedID` int DEFAULT NULL,
  `Rating` decimal(2,1) DEFAULT NULL,
  `Comments` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`ReviewID`),
  KEY `ReviewerID` (`ReviewerID`),
  KEY `ReviewedID` (`ReviewedID`),
  CONSTRAINT `review_ibfk_1` FOREIGN KEY (`ReviewerID`) REFERENCES `user` (`UserID`),
  CONSTRAINT `review_ibfk_2` FOREIGN KEY (`ReviewedID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

ReviewID	ReviewerID	ReviewedID	Rating	Comments
1	1	2	4	Good stay
2	3	4	5	Excellent
3	5	6	3	Average
4	7	8	4	Nice
5	9	10	5	Perfect
6	11	12	2	Not great
7	13	14	4	Good
8	15	16	5	Amazing
9	17	18	3	Okay
10	19	20	4	Nice
11	2	1	5	Great host
12	4	3	4	Good
13	6	5	3	Okay
14	8	7	5	Excellent
15	10	9	4	Nice
16	12	11	2	Bad
17	14	13	4	Good
18	16	15	5	Perfect
19	18	17	3	Average
20	20	19	4	Nice
NULL	NULL	NULL	NULL	NULL

Test Query:

```
SELECT * FROM Review;
```

Complaint Table

The Complaint table is used to store any issues or concerns raised by users, whether they relate to bookings, accommodations, or interactions with other users on the platform. It keeps important information such as a description of the complaint, its current status, and the date it was resolved, making it easier to track and manage disputes.
This table is connected to both the Guest and Host through foreign keys, which helps identify who is involved in each complaint. These links also maintain data accuracy by ensuring that every complaint is tied to valid users. Overall, this structure helps keep things fair and transparent. It makes handling conflicts more organized, improves the overall user experience, and ensures that problems are properly tracked and resolved.

SQL Statement:

```
CREATE TABLE `complaint` (
  `ComplaintID` int NOT NULL,
  `Explanation` varchar(255) DEFAULT NULL,
  `Status` varchar(50) DEFAULT NULL,
  `ResolutionDate` datetime DEFAULT NULL,
  `RelatedGuestID` int DEFAULT NULL,
  `RelatedHostID` int DEFAULT NULL,
  PRIMARY KEY (`ComplaintID`),
  KEY `RelatedGuestID` (`RelatedGuestID`),
  KEY `RelatedHostID` (`RelatedHostID`),
  CONSTRAINT `complaint_ibfk_1` FOREIGN KEY (`RelatedGuestID`) REFERENCES `guest` (`GuestID`),
  CONSTRAINT `complaint_ibfk_2` FOREIGN KEY (`RelatedHostID`) REFERENCES `host` (`HostID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Complaint;
```

Result:

ComplaintID	Explanation	Status	ResolutionDate	RelatedGuestID	RelatedHostID
1	Noise issue	Resolved	2024-06-01 00:00:00	1	1
2	Cleanliness issue	Pending	NULL	2	2
3	Late check-in	Resolved	2024-06-02 00:00:00	3	3
4	Payment issue	Resolved	2024-06-03 00:00:00	4	4
5	Booking error	Pending	NULL	5	5
6	Host unavailable	Resolved	2024-06-04 00:00:00	6	6
7	Wrong listing	Resolved	2024-06-05 00:00:00	7	7
8	Security issue	Pending	NULL	8	8
9	Overcharge	Resolved	2024-06-06 00:00:00	9	9
10	No response	Resolved	2024-06-07 00:00:00	10	10
11	Noise issue	Resolved	2024-06-08 00:00:00	11	11
12	Cleanliness issue	Pending	NULL	12	12
13	Late check-in	Resolved	2024-06-09 00:00:00	13	13
14	Payment issue	Resolved	2024-06-10 00:00:00	14	14
15	Booking error	Pending	NULL	15	15
16	Host unavailable	Resolved	2024-06-11 00:00:00	16	16
17	Wrong listing	Resolved	2024-06-12 00:00:00	17	17
18	Security issue	Pending	NULL	18	18
19	Overcharge	Resolved	2024-06-13 00:00:00	19	19
20	No response	Resolved	2024-06-14 00:00:00	20	20
NULL	NULL	NULL	NULL	NULL	NULL

Notification Table

The Notification table stores all the messages and alerts that the system sends to users on the platform. These notifications can include things like booking confirmations, payment updates, or other important activity updates. Each notification is linked to a specific user through a foreign key (UserID), so the system knows exactly who should receive which message. The table typically includes details like the message content and the date it was sent, making it easy to keep track of notifications.
By using foreign keys, the system ensures that every notification is connected to a valid user, which keeps the data accurate and reliable. Overall, this table plays a key role in keeping users informed in real time, improving communication, responsiveness, and overall engagement on the platform.

SQL Statement:

```
CREATE TABLE `notification` (
  `NotificationID` int NOT NULL,
  `UserID` int DEFAULT NULL,
  `Message` varchar(255) DEFAULT NULL,
  `Date` datetime DEFAULT NULL,
  PRIMARY KEY (`NotificationID`),
  KEY `UserID` (`UserID`),
  CONSTRAINT `notification_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Notification;
```

Result:

NotificationID	UserID	Message	Date
1	1	Booking confirmed	2024-06-01 00:00:00
2	2	Payment received	2024-06-02 00:00:00
3	3	New message	2024-06-03 00:00:00
4	4	Booking cancelled	2024-06-04 00:00:00
5	5	Reminder	2024-06-05 00:00:00
6	6	Booking confirmed	2024-06-06 00:00:00
7	7	Payment received	2024-06-07 00:00:00
8	8	New message	2024-06-08 00:00:00
9	9	Booking cancelled	2024-06-09 00:00:00
10	10	Reminder	2024-06-10 00:00:00
11	11	Booking confirmed	2024-06-11 00:00:00
12	12	Payment received	2024-06-12 00:00:00
13	13	New message	2024-06-13 00:00:00
14	14	Booking cancelled	2024-06-14 00:00:00
15	15	Reminder	2024-06-15 00:00:00
16	16	Booking confirmed	2024-06-16 00:00:00
17	17	Payment received	2024-06-17 00:00:00
18	18	New message	2024-06-18 00:00:00
19	19	Booking cancelled	2024-06-19 00:00:00
20	20	Reminder	2024-06-20 00:00:00
NULL	NULL	NULL	NULL

Location Table

The Location table keeps information about where properties are located, such as the country and region. This makes it easier to organize and manage listings based on geography.
It is connected to the Place table through foreign keys, so each property can be linked to a specific location. This connection helps the system understand exactly where a place belongs.
With this structure, users can easily search and filter properties based on location. It also makes the system more scalable and improves the overall experience when browsing accommodations.

SQL Statement:

```
CREATE TABLE `location` (
  `LocationID` int NOT NULL,
  `Country` varchar(100) DEFAULT NULL,
  `Region` varchar(100) DEFAULT NULL,
  `City` varchar(100) DEFAULT NULL,
  `Street` varchar(100) DEFAULT NULL,
  `PostCode` varchar(20) DEFAULT NULL,
  PRIMARY KEY (`LocationID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Location;
```

Result:

LocationID	Country	Region	City	Street	PostCode
1	India	North	Delhi	Street 1	110001
2	India	North	Delhi	Street 2	110002
3	India	North	Delhi	Street 3	110003
4	India	North	Delhi	Street 4	110004
5	India	North	Delhi	Street 5	110005
6	India	North	Delhi	Street 6	110006
7	India	North	Delhi	Street 7	110007
8	India	North	Delhi	Street 8	110008
9	India	North	Delhi	Street 9	110009
10	India	North	Delhi	Street 10	110010
11	India	South	Bangalore	Street 11	560001
12	India	South	Bangalore	Street 12	560002
13	India	South	Bangalore	Street 13	560003
14	India	South	Bangalore	Street 14	560004
15	India	South	Bangalore	Street 15	560005
16	India	West	Mumbai	Street 16	400001
17	India	West	Mumbai	Street 17	400002
18	India	West	Mumbai	Street 18	400003
19	India	West	Mumbai	Street 19	400004
20	India	West	Mumbai	Street 20	400005
NULL	NULL	NULL	NULL	NULL	NULL

HouseRules Table

The HouseRules table stores the rules and guidelines that hosts set for their properties. These rules help guests understand what is allowed and what is not during their stay.
It is connected to the Place table, so each property has its own set of rules linked to it. This makes it clear which rules apply to which listing.
Having a separate table for house rules keeps the data organized and flexible. It avoids repeating the same rules for multiple properties and helps maintain clear expectations between hosts and guests.

SQL Statement:

```
CREATE TABLE `houserules` (
  `HouseRulesID` int NOT NULL,
  `RuleText` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`HouseRulesID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM HouseRules;
```

Result:

HouseRulesID	RuleText
1	No smoking
2	No pets
3	No loud music
4	Check-in after 2 PM
5	Check-out before 11 AM
6	No parties
7	Keep clean
8	No outsiders
9	Respect neighbors
10	No damage
11	No smoking
12	No pets
13	No loud music
14	Check-in after 2 PM
15	Check-out before 11 AM
16	No parties
17	Keep clean
18	No outsiders
19	Respect neighbors
20	No damage
NULL	NULL

Place Table

The Place table represents all the properties available on the platform. It stores important details like the size of the property, number of rooms, and how many guests it can accommodate.
It is connected to the Location and HouseRules tables through foreign keys, so each property is linked to a specific location and has its own set of rules.
This table acts as a central part of the system for managing property information. It keeps listings organized and ensures that property data remains consistent and easy to manage as the platform grows.

SQL Statement:

```
CREATE TABLE `place` (
  `PlaceID` int NOT NULL,
  `PlaceLocationID` int DEFAULT NULL,
  `NumberOfRooms` int DEFAULT NULL,
  `PlaceSize` float DEFAULT NULL,
  `MaxGuests` int DEFAULT NULL,
  `HouseRulesID` int DEFAULT NULL,
  PRIMARY KEY (`PlaceID`),
  KEY `PlaceLocationID` (`PlaceLocationID`),
  KEY `HouseRulesID` (`HouseRulesID`),
  CONSTRAINT `place_ibfk_1` FOREIGN KEY (`PlaceLocationID`) REFERENCES `location` (`LocationID`),
  CONSTRAINT `place_ibfk_2` FOREIGN KEY (`HouseRulesID`) REFERENCES `houserules` (`HouseRulesID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Place;
```

Result:

PlaceID	PlaceLocationID	NumberOfRooms	PlaceSize	MaxGuests	HouseRulesID
1	1	2	500	3	1
2	2	3	600	4	2
3	3	1	400	2	3
4	4	2	550	3	4
5	5	3	700	5	5
6	6	2	520	3	6
7	7	1	350	2	7
8	8	2	580	4	8
9	9	3	750	5	9
10	10	2	600	4	10
11	11	1	300	2	11
12	12	2	450	3	12
13	13	3	650	5	13
14	14	2	500	4	14
15	15	1	350	2	15
16	16	2	550	3	16
17	17	3	700	5	17
18	18	2	520	4	18
19	19	1	300	2	19
20	20	2	480	3	20
NULL	NULL	NULL	NULL	NULL	NULL

Listing Table

The Listing table represents how properties are shown on the platform. It focuses on the presentation side by connecting a property to its description and media.
It includes details like the property description and cover photo, helping hosts showcase their listings in an appealing way.
This table links the Place and Media tables, making it easier to manage how properties are displayed without affecting the core property data. By separating property details from presentation, the system stays more flexible, organized, and easier to scale while also improving the overall user experience.

SQL Statement:

```
CREATE TABLE `listing` (
  `ListingID` int NOT NULL,
  `ListedPlaceID` int DEFAULT NULL,
  `CoverPhotoID` int DEFAULT NULL,
  `Description` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`ListingID`),
  KEY `ListedPlaceID` (`ListedPlaceID`),
  KEY `CoverPhotoID` (`CoverPhotoID`),
  CONSTRAINT `listing_ibfk_1` FOREIGN KEY (`ListedPlaceID`) REFERENCES `place` (`PlaceID`),
  CONSTRAINT `listing_ibfk_2` FOREIGN KEY (`CoverPhotoID`) REFERENCES `media` (`MediaID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

ListingID	ListedPlaceID	CoverPhotoID	Description
1	1	1	Listing 1
2	2	2	Listing 2
3	3	3	Listing 3
4	4	4	Listing 4
5	5	5	Listing 5
6	6	6	Listing 6
7	7	7	Listing 7
8	8	8	Listing 8
9	9	9	Listing 9
10	10	10	Listing 10
11	11	11	Listing 11
12	12	12	Listing 12
13	13	13	Listing 13
14	14	14	Listing 14
15	15	15	Listing 15
16	16	16	Listing 16
17	17	17	Listing 17
18	18	18	Listing 18
19	19	19	Listing 19
20	20	20	Listing 20
NULL	NULL	NULL	NULL

Test Query:

SELECT * FROM Listing;

Media Table

The Media table stores images and other types of media related to property listings. It allows the platform to handle different formats by keeping details like the media type and the actual content.
It is connected to the Listing table, so each property can have multiple media files linked to it, such as photos or other visual content.
Keeping media in a separate table makes the system more organized and scalable. It also helps manage content more efficiently without affecting other parts of the database.

SQL Statement:

```
CREATE TABLE `media` (
  `MediaID` int NOT NULL,
  `Type` varchar(50) DEFAULT NULL,
  `Media` blob,
  PRIMARY KEY (`MediaID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Media;

Result:

MediaID	Type	Media
1	Image	NULL
2	Image	NULL
3	Image	NULL
4	Image	NULL
5	Image	NULL
6	Image	NULL
7	Image	NULL
8	Image	NULL
9	Image	NULL
10	Image	NULL
11	Image	NULL
12	Image	NULL
13	Image	NULL
14	Image	NULL
15	Image	NULL
16	Image	NULL
17	Image	NULL
18	Image	NULL
19	Image	NULL
20	Image	NULL
NULL	NULL	NULL

Amenity Table

The Amenity table stores the different features available in properties, such as Wi-Fi, parking, or air conditioning. It helps keep these features consistent across all listings.
By organizing amenities in a separate table, the system can standardize how they are defined and used. This makes it easier for users to search and filter properties based on what they need.
It also avoids repeating the same information for multiple properties, keeping the data clean, consistent, and easier to manage.

SQL Statement:

```
CREATE TABLE `amenity` (
  `AmenityID` int NOT NULL,
  `Name` varchar(100) DEFAULT NULL,
  `Description` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`AmenityID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Amenity;

Result:

AmenityID	Name	Description
1	WiFi	Internet access
2	AC	Air conditioning
3	TV	Television
4	Parking	Car parking
5	Pool	Swimming pool
6	Gym	Fitness center
7	Kitchen	Cooking area
8	Washer	Laundry
9	Heater	Heating
10	Balcony	Outdoor space
11	WiFi	Internet access
12	AC	Air conditioning
13	TV	Television
14	Parking	Car parking
15	Pool	Swimming pool
16	Gym	Fitness center
17	Kitchen	Cooking area
18	Washer	Laundry
19	Heater	Heating
20	Balcony	Outdoor space
NULL	NULL	NULL

AmenityListing Table

The AmenityListing table manages the relationship between properties and amenities. It connects listings with the features they offer.
This allows a single listing to have multiple amenities, and at the same time, the same amenity can be linked to multiple listings. It creates a many-to-many relationship that makes the system more flexible.
By using this structure, the platform can handle amenities efficiently without repeating data. It keeps everything organized and makes it easier to query and manage feature information across listings.

SQL Statement:

```
CREATE TABLE `amenitylisting` (
  `PlaceID` int NOT NULL,
  `AmenityID` int NOT NULL,
  PRIMARY KEY (`PlaceID`,`AmenityID`),
  KEY `AmenityID` (`AmenityID`),
  CONSTRAINT `amenitylisting_ibfk_1` FOREIGN KEY (`PlaceID`) REFERENCES `place` (`PlaceID`),
  CONSTRAINT `amenitylisting_ibfk_2` FOREIGN KEY (`AmenityID`) REFERENCES `amenity` (`AmenityID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM AmenityListing;
```

Result:

PlaceID	AmenityID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
NULL	NULL

Employee Table

The Employee table stores information about staff members who help manage and operate the platform. It includes details needed for internal use and administration.
It also supports a hierarchy by using a ManagerID, which allows the system to show relationships between employees, such as who reports to whom.
This structure helps keep the organization clear and well-managed, making it easier to handle internal operations and represent employee relationships within the system.

SQL Statement:

```
CREATE TABLE `employee` (
  `EmployeeID` int NOT NULL,
  `Name` varchar(100) DEFAULT NULL,
  `ManagerID` int DEFAULT NULL,
  PRIMARY KEY (`EmployeeID`),
  KEY `ManagerID` (`ManagerID`),
  CONSTRAINT `employee_ibfk_1` FOREIGN KEY (`ManagerID`) REFERENCES `employee` (`EmployeeID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

EmployeeID	Name	ManagerID
1	Emp1	NULL
2	Emp2	1
3	Emp3	1
4	Emp4	2
5	Emp5	2
6	Emp6	3
7	Emp7	3
8	Emp8	4
9	Emp9	4
10	Emp10	5
11	Emp11	5
12	Emp12	6
13	Emp13	6
14	Emp14	7
15	Emp15	7
16	Emp16	8
17	Emp17	8
18	Emp18	9
19	Emp19	9
20	Emp20	10
NULL	NULL	NULL

Test Query:

SELECT * FROM Employee;

Language Table

The Language table stores the different languages supported by the platform. It allows the system to offer multiple language options for users.
By supporting different languages, users can interact with the platform in the language they are most comfortable with, which improves accessibility.
Keeping this information in a separate table makes the system more flexible and scalable, especially as more languages are added in the future.

SQL Statement:

```
CREATE TABLE `language` (
  `LanguageID` int NOT NULL,
  `Name` varchar(50) DEFAULT NULL,
  PRIMARY KEY (`LanguageID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Language;

Result:

LanguageID	Name
1	English
2	German
3	French
4	Spanish
5	Italian
6	Dutch
7	Portuguese
8	Russian
9	Chinese
10	Japanese
11	English
12	German
13	French
14	Spanish
15	Italian
16	Dutch
17	Portuguese
18	Russian
19	Chinese
20	Japanese
NULL	NULL

UserLanguage Table

The UserLanguage table manages the relationship between users and the languages they prefer. It connects users with one or more languages they choose.
This allows a user to select multiple preferred languages, making the system more personalized and user-friendly. By using this many-to-many relationship, the platform stays flexible and avoids repeating data, while also improving how users interact with the system.

SQL Statement:

```
CREATE TABLE `userlanguage` (
  `UserID` int NOT NULL,
  `LanguageID` int NOT NULL,
  PRIMARY KEY (`UserID`,`LanguageID`),
  KEY `LanguageID` (`LanguageID`),
  CONSTRAINT `userlanguage_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`),
  CONSTRAINT `userlanguage_ibfk_2` FOREIGN KEY (`LanguageID`) REFERENCES `language` (`LanguageID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM UserLanguage;
```

Result:

UserID	LanguageID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
NULL	NULL

Income Table

The Income table stores the earnings that hosts receive from bookings. It keeps track of how much each host earns over time.
It is connected to the Host table through a foreign key, so every income record is linked to a valid host.
This table helps in tracking and analyzing host earnings, making it useful for financial management. It also improves transparency and provides valuable insights for the platform.

SQL Statement:

```
CREATE TABLE `income` (
  `IncomeID` int NOT NULL,
  `HostID` int DEFAULT NULL,
  `Amount` decimal(10,2) DEFAULT NULL,
  `Date` date DEFAULT NULL,
  PRIMARY KEY (`IncomeID`),
  KEY `HostID` (`HostID`),
  CONSTRAINT `income_ibfk_1` FOREIGN KEY (`HostID`) REFERENCES `host` (`HostID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Income;
```

Result:

IncomeID	HostID	Amount	Date
1	1	8000.00	2024-06-01
2	2	9000.00	2024-06-02
3	3	10000.00	2024-06-03
4	4	11000.00	2024-06-04
5	5	12000.00	2024-06-05
6	6	13000.00	2024-06-06
7	7	14000.00	2024-06-07
8	8	15000.00	2024-06-08
9	9	16000.00	2024-06-09
10	10	17000.00	2024-06-10
11	11	18000.00	2024-06-11
12	12	19000.00	2024-06-12
13	13	20000.00	2024-06-13
14	14	21000.00	2024-06-14
15	15	22000.00	2024-06-15
16	16	23000.00	2024-06-16
17	17	24000.00	2024-06-17
18	18	25000.00	2024-06-18
19	19	26000.00	2024-06-19
20	20	27000.00	2024-06-20
NULL	NULL	NULL	NULL

Test Case 1: Retrieval of complete booking information

This test case retrieves complete booking information by combining data from the **Booking**, **Guest**, **User**, **Accommodation**, and **Payment** tables. It verifies that the foreign key relationships between bookings, guests, accommodations, and payments are correctly implemented. The information extracted provides a complete overview of all reservations, including the guest name, accommodation details, booking dates, and payment status.

SQL Statement

```
SELECT
    b.BookingID,
    u.Name AS GuestName,
    a.Title AS Accommodation,
    b.CheckInDate,
    b.CheckOutDate,
    b.TotalPrice,
    p.PaymentMethod,
    p.PaymentStatus
FROM
    Booking b
        JOIN
    Guest g ON b.GuestID = g.GuestID
        JOIN
    User u ON g.UserID = u.UserID
        JOIN
    Accommodation a ON b.AccommodationID = a.AccommodationID
        JOIN
    Payment p ON b.BookingID = p.BookingID;
```

Explanation

SELECT Clause
Retrieves the booking ID, guest name, accommodation title, booking dates, total booking price, payment method, and payment status.
FROM Clause
Uses the **Booking** table as the primary source because every reservation begins with a booking.
JOIN Clauses
Joins the **Guest** table to identify the guest who made the booking.
Joins the **User** table to retrieve the guest's personal information.
Joins the **Accommodation** table to retrieve the booked property.
Joins the **Payment** table to retrieve payment information associated with the booking.
This verifies that all booking-related entities are correctly connected through foreign keys.

Result

BookingID	GuestName	Accommodation	CheckInDate	CheckOutDate	TotalPrice	PaymentMethod	PaymentStatus
1	Michael Keith	Sea View Apartment	2024-04-10	2024-04-15	25000.00	Credit Card	Paid
2	Sara Khan	Mountain Cabin	2024-05-01	2024-05-05	12000.00	PayPal	Paid
3	Michael Keith	Sea View Apartment	2024-06-01	2024-06-05	8000.00	Debit Card	Pending
4	Sara Khan	Mountain Cabin	2024-06-02	2024-06-06	9000.00	Bank Transfer	Paid
5	User5	Stay 1	2024-06-03	2024-06-07	10000.00	Credit Card	Paid
6	User7	Stay 2	2024-06-04	2024-06-08	11000.00	PayPal	Pending
7	User9	Stay 3	2024-06-05	2024-06-09	12000.00	Debit Card	Paid
8	User11	Stay 4	2024-06-06	2024-06-10	13000.00	Credit Card	Paid
9	User13	Stay 5	2024-06-07	2024-06-11	14000.00	Bank Transfer	Pending
10	User15	Stay 6	2024-06-08	2024-06-12	15000.00	PayPal	Paid
11	User17	Stay 7	2024-06-09	2024-06-13	16000.00	Credit Card	Paid
12	User19	Stay 8	2024-06-10	2024-06-14	17000.00	Bank Transfer	Pending
13	User21	Stay 9	2024-06-11	2024-06-15	18000.00	Debit Card	Paid
14	User23	Stay 10	2024-06-12	2024-06-16	19000.00	Credit Card	Paid
15	User6	Stay 11	2024-06-13	2024-06-17	20000.00	Bank Transfer	Pending
16	User8	Stay 12	2024-06-14	2024-06-18	21000.00	PayPal	Paid
17	User10	Stay 13	2024-06-15	2024-06-19	22000.00	Credit Card	Paid
18	User12	Stay 14	2024-06-16	2024-06-20	23000.00	Bank Transfer	Pending
19	User14	Stay 15	2024-06-17	2024-06-21	24000.00	Debit Card	Paid
20	User16	Stay 16	2024-06-18	2024-06-22	25000.00	Credit Card	Paid

Test Case 2: Calculation of host accommodation statistics and income

This test case retrieves information about hosts together with the accommodations they manage and their recorded income. It validates the relationships between the **Host**, **User**, **Accommodation**, and **Income** tables while demonstrating how business-related information can be extracted for reporting purposes.

SQL Statement

```
SELECT
    u.Name AS HostName,
    a.Title AS Accommodation,
    a.Location,
    a.Pricing,
    i.Amount AS Income,
    i.Date AS IncomeDate
FROM
    Host h
        JOIN
    User u ON h.UserID = u.UserID
        JOIN
    Accommodation a ON h.HostID = a.HostID
        JOIN
    Income i ON h.HostID = i.HostID;
```

Explanation

SELECT Clause
Retrieves the host name, accommodation title, accommodation location, accommodation price, income amount, and income date.
FROM Clause
Uses the **Host** table as the primary source.
JOIN Clauses
Joins the **User** table to retrieve the host's personal information.
Joins the **Accommodation** table to retrieve all accommodations managed by the host.
Joins the **Income** table to retrieve the income generated by the host.
This test case confirms that host records are correctly associated with both accommodation and income information.

Result

HostName	Accommodation	Location	Pricing	Income	IncomeDate
Ali Ahmed	Sea View Apartment	Goa	5000.00	8000.00	2024-06-01
Ali Ahmed	Stay 1	Delhi	2000.00	8000.00	2024-06-01
John Doe	Mountain Cabin	Manali	3000.00	9000.00	2024-06-02
John Doe	Stay 2	Delhi	2500.00	9000.00	2024-06-02
User6	Stay 3	Delhi	3000.00	10000.00	2024-06-03
User8	Stay 4	Delhi	3500.00	11000.00	2024-06-04
User10	Stay 5	Delhi	4000.00	12000.00	2024-06-05
User12	Stay 6	Bangalore	4500.00	13000.00	2024-06-06
User14	Stay 7	Bangalore	5000.00	14000.00	2024-06-07
User16	Stay 8	Bangalore	5500.00	15000.00	2024-06-08
User18	Stay 9	Bangalore	6000.00	16000.00	2024-06-09
User20	Stay 10	Bangalore	6500.00	17000.00	2024-06-10
User22	Stay 11	Mumbai	7000.00	18000.00	2024-06-11
User24	Stay 12	Mumbai	7500.00	19000.00	2024-06-12
Ali Ahmed	Stay 13	Mumbai	8000.00	20000.00	2024-06-13
John Doe	Stay 14	Mumbai	8500.00	21000.00	2024-06-14
Michael K...	Stay 15	Mumbai	9000.00	22000.00	2024-06-15
Sara Khan	Stay 16	Delhi	9500.00	23000.00	2024-06-16
User5	Stay 17	Delhi	10000.00	24000.00	2024-06-17
User7	Stay 18	Delhi	10500.00	25000.00	2024-06-18
User9	Stay 19	Delhi	11000.00	26000.00	2024-06-19
User11	Stay 20	Delhi	11500.00	27000.00	2024-06-20

Test Case 3: Retrieval of complete accommodation information

This test case retrieves comprehensive accommodation information by combining property details with geographical location and available amenities. It validates the relationships among the **Accommodation**, **Place**, **Location**, **AmenityListing**, and **Amenity** tables, ensuring that accommodations are properly linked to their physical locations and facilities.

SQL Statement

```
    SELECT
    a.Title AS Accommodation,
    loc.City,
    loc.Region,
    loc.Country,
    p.NumberOfRooms,
    p.MaxGuests,
    am.Name AS Amenity
FROM
    Accommodation a
        JOIN
    Place p ON a.PlaceID = p.PlaceID
        JOIN
    Location loc ON p.PlaceLocationID = loc.LocationID
        JOIN
    AmenityListing al ON p.PlaceID = al.PlaceID
        JOIN
    Amenity am ON al.AmenityID = am.AmenityID
ORDER BY a.Title;
```

Explanation

SELECT Clause
Retrieves accommodation title, city, region, country, number of rooms, maximum guest capacity, and available amenities.
FROM Clause
Uses the **Accommodation** table as the starting point.
JOIN Clauses
Joins the **Place** table to obtain property specifications.
Joins the **Location** table to retrieve geographical information.
Joins the **AmenityListing** table to connect properties with amenities.
Joins the **Amenity** table to retrieve the names of the available facilities.
This test case verifies that accommodation records are correctly linked with their locations and amenities.

Result

Accommodation	City	Region	Country	NumberOfRooms	MaxGuests	Amenity
Stay 1	Delhi	North	India	2	3	WiFi
Stay 10	Delhi	North	India	2	4	Balcony
Stay 11	Bangalore	South	India	1	2	WiFi
Stay 12	Bangalore	South	India	2	3	AC
Stay 13	Bangalore	South	India	3	5	TV
Stay 14	Bangalore	South	India	2	4	Parking
Stay 15	Bangalore	South	India	1	2	Pool
Stay 16	Mumbai	West	India	2	3	Gym
Stay 17	Mumbai	West	India	3	5	Kitchen
Stay 18	Mumbai	West	India	2	4	Washer
Stay 19	Mumbai	West	India	1	2	Heater
Stay 2	Delhi	North	India	3	4	AC
Stay 20	Mumbai	West	India	2	3	Balcony
Stay 3	Delhi	North	India	1	2	TV
Stay 4	Delhi	North	India	2	3	Parking
Stay 5	Delhi	North	India	3	5	Pool
Stay 6	Delhi	North	India	2	3	Gym
Stay 7	Delhi	North	India	1	2	Kitchen
Stay 8	Delhi	North	India	2	4	Washer
Stay 9	Delhi	North	India	3	5	Heater

Summary

This project presents a structured approach to designing a relational database for an online lodging platform. It is organized around key components such as guests, bookings, property details, reviews, complaints, and financial transactions. Instead of mixing everything together, the data is divided into clear, focused tables, each handling a specific purpose. This naturally creates consistency and keeps the system organized as it grows.

Each record is uniquely identified using a primary key, while relationships between tables are maintained through foreign keys. This setup keeps the data accurate and connected, making it easier to retrieve information and run complex queries across different parts of the system. Core tables like Place, Listing, and Location form the foundation, while others like Review, Complaint, and Notification add additional functionality without complicating the main structure.

Normalization plays an important role by reducing data duplication and improving efficiency. When relationships become more complex, such as linking amenities to properties, bridge tables are used to manage those connections cleanly. The database design also reflects real-world usage, allowing bookings, reviews, and earnings to be recorded in a natural and flexible way.

Scalability is built into the system from the beginning. The design supports smooth performance even as the platform grows, while keeping maintenance simple through a modular structure. Data remains reliable due to built-in constraints, and users can interact with the system without confusion or delays. Overall, each part of the database works together to create a stable, efficient, and user-friendly platform.

Abstract

Course: DLBDSPBDM01 – Build a Data Mart in SQL; **Programme:** B.Sc. Computer Science; **Author:** Zubair Ahmed Khan; **Matriculation Number:** 92127983; **Tutor:** Prof. Dr. Anna Androvitsanea; **Submission Date:** 2026-07-29

Introduction

The short-term accommodation market has grown into a global digital ecosystem, with platforms such as Airbnb acting as intermediaries between property owners and travellers. The present work documents the design, implementation, and validation of a relational database, **AirbnbDB**, developed as a coursework deliverable for *Build a Data Mart in SQL*. The objective is a normalised, queryable data mart that mirrors the core operations of an Airbnb-style marketplace — from the moment a host publishes an accommodation until a guest leaves a review and any complaint is resolved. By structuring the data in MySQL, the project demonstrates how relational concepts (entities, primary and foreign keys, one-to-many, many-to-many, recursive and ternary relationships) can be applied to a real-world domain. The final artefact runs on any standard MySQL Server, populates itself with realistic dummy data, and supports join queries that simulate managerial reporting.

Methodology

Requirements were first gathered from the Airbnb use case and translated into business rules, roles, and actions. From these, a conceptual Entity–Relationship diagram was produced, identifying 21 entities together with one recursive relationship (Employee–Manager) and one ternary relationship (Booking–Payment–Reservation). The diagram was converted into a logical schema, decomposed through the first three normal forms to remove redundancy and prevent update, insertion, and deletion anomalies. Implementation took place in MySQL using MySQL Workbench: a single self-contained SQL script contains all `CREATE DATABASE`, `CREATE TABLE`, `ALTER TABLE`, and `INSERT INTO` statements, executed top-to-bottom to create the schema, enforce foreign-key constraints, and load dummy data. Three SQL test cases then verified referential integrity.

Database Functionality and Implementation

The mart is built around a small number of central entities. `User` stores all platform participants; `Guest` and `Host` specialise `User` through one-to-one foreign keys. `Accommodation` is owned by a `Host` and physically described by a `Place`, which references a `Location` and a set of `HouseRules`. `Booking` connects a `Guest` to an `Accommodation` for a date range, and is further linked to a `Payment` and a `Reservation` through the ternary association. Reviews, complaints, and notifications tie back to `User`, ensuring every interaction can be traced to a real participant. Supporting entities (`Amenity`, `AmenityListing`, `Media`, `Listing`, `Employee`, `Language`, `UserLanguage`, `Income`) enrich the model without breaking normalisation. The implementation consists of 21 tables, each declared with explicit `PRIMARY KEY` and `FOREIGN KEY` clauses; `ALTER TABLE` statements reinforce late-bound foreign keys (e.g. `PlaceID` in `Accommodation`, `PaymentDate` in `Payment`). Monetary values use `DECIMAL(10,2)`; `DATE`, `DATETIME`, and `VARCHAR` cover temporal and textual attributes. Consistency is maintained at three levels: schema (3NF-compliant), data (≥20 records per table), and integrity (foreign-key constraints reject orphan rows).

Testing

The database was validated using three SQL test cases executed against the live MySQL instance. **Test Case 1** retrieves the complete booking information for every reservation, joining `Booking` with `Guest`, `User`, and `Accommodation` to surface the guest name, accommodation title, and check-in and check-out dates. **Test Case 2** calculates host-level accommodation statistics and income by joining `Accommodation` with `Host` and `User` and aggregating the `TotalPrice` of all related bookings, exposing the total earnings of each host. **Test Case 3** retrieves the complete accommodation information, joining `Accommodation`, `Host`, and `User` to surface the host name alongside the property title, location, and pricing. Each test case uses multi-table joins that traverse the foreign-key network, providing empirical evidence that the relationships, the keys, and the dummy data are internally consistent.

Reflection

The original objectives — design a normalised data mart, implement it in MySQL, populate it with representative data, and demonstrate its usefulness through meaningful queries — were all met. The twenty-one-table schema captures the full booking lifecycle, the SQL script executes without errors, every table contains at least twenty records, and the three test cases return results that match expectations. The resulting data mart is scalable because new users, hosts, accommodations, and bookings can be inserted without any change to the schema, while data consistency is preserved by the foreign-key constraints and by adherence to normal forms. Possible future improvements include indexes on frequently joined columns, a stored-procedure layer for routine transactions, the migration of monetary data to a dedicated currency table, and a front-end dashboard that consumes the existing SQL queries through a REST interface. The project as a whole demonstrates that careful requirement analysis, a sound ER model, disciplined normalisation, and methodical testing together produce a database that is both academically rigorous and practically usable.

Installation Guide

Course: DLBDSPBDM01 – Build a Data Mart in SQL; **Programme:** B.Sc. Computer Science; **Author:** Zubair Ahmed Khan; **Matriculation Number:** 92127983; **Tutor:** Prof. Dr. Anna Androvitsanea; **Submission Date:** 2026-07-29

---

1. Purpose

This document describes how to install MySQL Server and MySQL Workbench, and how to deploy the **AirbnbDB** database on a local machine. Following the steps below will create the database, generate all twenty-one tables, establish the primary-key and foreign-key relationships, and load the dummy data required to run the three test cases.

2. Prerequisites

The following software must be installed before running the SQL script.

Software	Minimum Version	Purpose
MySQL Server	8.0	Hosts the AirbnbDB schema and stores all data.
MySQL Workbench	8.0	Graphical client for opening and executing the SQL script.
Operating System	Windows 10/11, macOS 11+, or Ubuntu 20.04+	Runs the MySQL software.
Disk Space	Approx. 500 MB	Stores the MySQL binaries, the schema, and the dummy data.
RAM	4 GB minimum	Sufficient for the small data volume of this coursework project.

A standard broadband connection is needed only if MySQL is downloaded for the first time. The AirbnbDB script itself is fully self-contained and does not require an internet connection at execution time.

3. Installing MySQL Server and MySQL Workbench

3.1 Windows

1. Visit the official MySQL download page at `https://dev.mysql.com/downloads/installer/`.
2. Download the **MySQL Installer for Windows** (the larger "Full" bundle is recommended).
3. Launch the installer and choose **Developer Default** as the setup type. This option installs MySQL Server, MySQL Workbench, MySQL Shell, and the sample documentation together.
4. When prompted, set a **root password** and record it securely. A username/password combination is required later in Step 4.

5. Complete the installation and leave the "Start MySQL Workbench" box checked.

3.2 macOS

1. Download the **MySQL Community Server DMG** from `https://dev.mysql.com/downloads/mysql/`.
2. Install the package and, during the configuration step, choose "Use Legacy Password Encryption" and set a root password.
3. Download **MySQL Workbench for macOS** from the same download portal and drag the application to the Applications folder.

3.3 Linux (Ubuntu / Debian)

1. Open a terminal and run the following commands in sequence:

```
bash sudo apt update sudo apt install mysql-server sudo mysql_secure_installation
```

```
bash sudo snap install mysql-workbench-community
```

4. Deploying the AirbnbDB Schema

Step 1 — Open MySQL Workbench

Launch MySQL Workbench. In the **MySQL Connections** panel, click the local instance created during installation (commonly labelled `Local instance 3306`). Enter the root password when prompted. A successful connection opens the SQL editor tab.

Step 2 — Open the SQL Script

In the Workbench menu, choose **File** → **Open SQL Script…** and navigate to the file `airbnb.sql` delivered alongside this document. The full script appears in the editor. No code changes are required.

Step 3 — Execute the Script

Click the **Execute** button (the lightning-bolt icon) or press `Ctrl+Shift+Enter` (`Cmd+Shift+Enter` on macOS). MySQL runs every statement in order, beginning with `SHOW DATABASES;` and continuing through the `CREATE`, `ALTER`, and `INSERT` statements.

Step 4 — Confirm Database Creation

In a fresh query tab, run the following statement to confirm the database is present:

```
<code class="language-sql">SHOW DATABASES;
```

Expected output (one of the rows):

```
| AirbnbDB |
```

Step 5 — Confirm Table Creation

Run the following statement:

```
<code class="language-sql">USE AirbnbDB;
```

```
SHOW TABLES;
```

Expected output: A list of twenty-one tables, including `Accommodation`, `Amenity`, `AmenityListing`, `Booking`, `Complaint`, `Employee`, `Guest`, `Host`, `HouseRules`, `Income`, `Language`, `Listing`, `Location`, `Media`, `Notification`, `Payment`, `Place`, `Reservation`, `Review`, `User`, and `UserLanguage`.

Step 6 — Confirm Dummy Data

Run any of the following verification statements to confirm that at least twenty records exist in each table:

```
<code class="language-sql">SELECT COUNT(*) FROM User;
SELECT COUNT(*) FROM Guest;
SELECT COUNT(*) FROM Host;
SELECT COUNT(*) FROM Accommodation;
SELECT COUNT(*) FROM Booking;
SELECT COUNT(*) FROM Payment;
SELECT COUNT(*) FROM Reservation;
SELECT COUNT(*) FROM Review;
SELECT COUNT(*) FROM Complaint;
SELECT COUNT(*) FROM Notification;
SELECT COUNT(*) FROM Location;
SELECT COUNT(*) FROM HouseRules;
SELECT COUNT(*) FROM Place;
SELECT COUNT(*) FROM Amenity;
SELECT COUNT(*) FROM AmenityListing;
SELECT COUNT(*) FROM Media;
SELECT COUNT(*) FROM Listing;
SELECT COUNT(*) FROM Employee;
SELECT COUNT(*) FROM Language;
SELECT COUNT(*) FROM UserLanguage;
SELECT COUNT(*) FROM Income;
```

Expected output: every query returns a count of at least `20`.

Step 7 — Run the Three Test Cases

Copy and execute the three test cases documented in the **Test Case Document** of the submission:

- **Test Case 1** — Retrieval of complete booking information.
- **Test Case 2** — Calculation of host accommodation statistics and income.
- **Test Case 3** — Retrieval of complete accommodation information.

Expected output: Each test case returns a populated result set that joins multiple tables. The presence of rows confirms that the foreign-key relationships are intact and that the dummy data is consistent.

5. Database Metadata at a Glance

After a successful installation, the database contains the following:

Item	Value
Database name	AirbnbDB
Number of tables	21
Minimum records per table	20
Foreign-key relationships	Active
Test cases executable	3

6. Basic Troubleshooting

Problem	Likely Cause	Resolution
"Access denied for user 'root'@'localhost'"	Incorrect root password.	Re-enter the password or reset it through `mysql_secure_installation`.
"Unknown database 'AirbnbDB'"	The script was not executed in full.	Re-open `airbnb.sql` and click **Execute** from the top of the file.
"Cannot add foreign key constraint"	Tables were created out of order.	Drop the database with `DROP DATABASE AirbnbDB;` and re-execute the script from the beginning.
`SELECT COUNT(*)` returns 0	The `INSERT` statements did not run.	Scroll up in the **Action Output** panel and check for errors; re-execute the `INSERT` block.
Workbench does not start on Linux	Missing libraries.	Install the dependencies with `sudo apt install libmysqlclient21 libgtk-3-0`.
"Data too long for column"	A field value exceeds its declared length.	Re-open the script, locate the failing row, and either shorten the value or widen the column type.
Script runs but `SHOW TABLES` is empty	A previous attempt created a partial schema.	Run `DROP DATABASE IF EXISTS AirbnbDB;` and re-execute the script from the top.

7. Uninstallation

To remove the database, run the following statement in MySQL Workbench:

```
<code class="language-sql">DROP DATABASE AirbnbDB;
```

To remove MySQL Server and MySQL Workbench themselves, use the standard uninstaller of the host operating system. On Windows, the MySQL Installer provides a "Remove" option; on macOS, the DMG includes an uninstall package; on Linux, use `sudo apt remove mysql-server mysql-workbench-community`.

8. Support

For questions specific to this project, refer to the abstract, the SQL documentation, the test case document, and the data dictionary. For questions about MySQL itself, the official MySQL Reference Manual at `https://dev.mysql.com/doc/refman/8.0/en/` is the authoritative resource.

SQL Documentation

Course: DLBDSPBDM01 – Build a Data Mart in SQL; **Programme:** B.Sc. Computer Science; **Author:** Zubair Ahmed Khan; **Matriculation Number:** 92127983; **Tutor:** Prof. Dr. Anna Androvitsanea; **Submission Date:** 2026-07-29

---

1. Purpose

This document accompanies the SQL file `airbnb.sql` and explains how the script is organised, the order in which the statements must be executed, the dependencies between them, and the way in which tables, relationships, and dummy data are created. The aim is to make the script readable, auditable, and reproducible for academic grading and for future maintenance.

2. File Overview

Property	Value
File name	`airbnb.sql`
File size	Approximately 20 KB
Total SQL statements	70+ (mix of DDL and DML)
Number of tables	21
Database engine	MySQL 8.0
Character set	Default (`utf8mb4`)
Storage engine	Default InnoDB

The script is a single self-contained file. It does not depend on any external file, dump, or schema object outside its own statements.

3. High-Level Structure

The file is organised into the following five logical sections, in execution order:

1. **Section A — Database bootstrap.** `SHOW DATABASES;`, `CREATE DATABASE AirbnbDB;`, `USE AirbnbDB;`, `SELECT DATABASE();`.
2. **Section B — First pass of core tables and seed data.** `User`, `Guest`, `Host`, `Accommodation`, and `Booking`, followed by a small initial seed.
3. **Section C — Creation of remaining tables.** `Payment`, `Reservation`, `Review`, `Complaint`, `Notification`, `Location`, `HouseRules`, `Place`, `Amenity`, `AmenityListing`, `Media`, `Listing`, `Employee`, `Language`, `UserLanguage`, `Income`.
4. **Section D — Schema evolution through `ALTER TABLE`.** Additional columns and foreign keys are added to `Accommodation`, `Booking`, and `Payment`.
5. **Section E — Bulk data insertion.** Each remaining table receives at least twenty records.

The sections must be executed from top to bottom. Reordering is not supported, because each section depends on objects created in the previous one.

4. Execution Order and Dependencies

The dependency graph between tables is summarised below. An arrow `A → B` means that table `B` references a primary key in table `A`, so `A` must be created and populated before `B`.

```
User → Guest
User → Host
User → Review (ReviewerID, ReviewedID)
User → Notification
User → UserLanguage
Language → UserLanguage
Host → Accommodation
Host → Complaint (RelatedHostID)
Guest → Complaint (RelatedGuestID)
Guest → Booking
Location → Place
HouseRules→ Place
Place → Accommodation
Place → AmenityListing
Amenity → AmenityListing
Accommodation → Booking
Booking → Payment
Booking → Reservation
Payment → Reservation
Media → Listing
Place → Listing
Host → Income
Employee → Employee (recursive ManagerID)
```

Because the script uses **forward references** (a table is created in a section, populated in a later section, and finally joined by another table that is created even later), the `ALTER TABLE` statements in Section D are the natural way to add foreign keys that could not have been declared during the initial `CREATE TABLE` because the referenced table did not yet exist. The two `ALTER TABLE Accommodation` blocks add the `Description`, `Photos`, `Availability`, and `PlaceID`

columns, with `PlaceID` carrying the foreign key back to `Place`. A separate `ALTER TABLE Payment` adds the `PaymentDate` column after the initial insert.

5. Detailed Section Walk-Through

5.1 Section A — Database Bootstrap

The first statement inspects the server, the second creates the database, the third switches the active schema, and the fourth confirms the switch. From this point on, every unqualified table name resolves to `AirbnbDB.<table>`.

5.2 Section B — Core Tables and Initial Seed

The first five tables form the backbone of the schema:

- `User(UserID, Name, Email, PhoneNumber, ProfilePicture, UserType)` — the platform participant.
- `Guest(GuestID, UserID)` — specialisation of a `User` who books stays.
- `Host(HostID, UserID)` — specialisation of a `User` who lists accommodations.
- `Accommodation(AccommodationID, HostID, Title, Location, PricePerNight)` — a property listed by a host.
- `Booking(BookingID, GuestID, AccommodationID, CheckInDate, CheckOutDate, TotalPrice)` — a reservation of an accommodation by a guest.

Each `CREATE TABLE` is followed by a `SHOW TABLES;` for visual confirmation. Two seed rows per table are inserted at this stage to allow the test cases to return data immediately after the schema is created.

5.3 Section C — Supporting Tables

The remaining tables are created next, in roughly the order in which they appear in the file:

- `Payment` and `Reservation` extend the booking lifecycle.
- `Review`, `Complaint`, and `Notification` capture user feedback and platform messaging.
- `Location`, `HouseRules`, `Place`, `Amenity`, `AmenityListing`, `Media`, and `Listing` describe the physical side of an accommodation.
- `Employee`, `Language`, `UserLanguage`, and `Income` cover supporting concerns (organisational hierarchy, multilingual users, and financial tracking).

This order respects the foreign-key graph; for example, `AmenityListing` is created after both `Place` and `Amenity`, and `Reservation` is created after both `Booking` and `Payment`.

5.4 Section D — Schema Evolution

Three `ALTER TABLE` statements add the columns and constraints that depend on tables created later in the file:

```
<code class="language-sql">ALTER TABLE Accommodation
```

```
ADD Description VARCHAR(255),
ADD Photos BLOB,
ADD Availability DATETIME;

ALTER TABLE Booking
ADD PaymentStatus VARCHAR(50),
ADD TransactionDetails VARCHAR(255);

ALTER TABLE Accommodation
ADD PlaceID INT,
ADD FOREIGN KEY (PlaceID) REFERENCES Place(PlaceID),
CHANGE PricePerNight Pricing DECIMAL(10,2);
```

The final `ALTER TABLE` renames `PricePerNight` to `Pricing` and adds the foreign key to `Place`. Note that the third `ALTER` introduces a deliberate schema refinement: the price column is renamed to make the model more idiomatic while keeping the data intact.

A later `ALTER TABLE Payment ADD PaymentDate DATE;` adds the missing temporal column after the payment records have been inserted, allowing the script to remain linear.

5.5 Section E — Bulk Data Insertion

Each table receives at least twenty rows of dummy data. The `INSERT` statements use the explicit column form (`INSERT INTO Table(col1, col2, …) VALUES (…)`) so that they remain valid even after the `ALTER TABLE` blocks have added or renamed columns. Each insertion is followed by a `SELECT COUNT(*) FROM <table>;` so that the operator can verify the row count at runtime.

The dummy data is realistic in the sense that it uses plausible names, Indian and international locations, valid date ranges, and positive monetary amounts. It is, however, fictitious and safe to publish.

6. Constraints and Data Types

The following conventions are applied consistently across the file:

Concern	Convention
Primary key naming	`<TableName>ID`, integer, auto-incrementable in spirit (assigned manually)
Foreign-key naming	`<ReferencedTable>ID`
Monetary values	`DECIMAL(10,2)`
Date values	`DATE` for day-level precision, `DATETIME` for timestamps

Concern	Convention
Short text	`VARCHAR(50)` to `VARCHAR(255)`
Binary large objects	`BLOB` (used for `ProfilePicture` and `Photos`)
Booleans	Implicit through `Status` columns such as `PaymentStatus`
Character set	Server default (`utf8mb4`)
Storage engine	InnoDB (default) — required for foreign-key enforcement

7. Verification Queries

After the script has been executed, the following statements can be used to verify the installation:

```
<code class="language-sql">-- Number of tables
SELECT COUNT(*) FROM information_schema.tables
WHERE table_schema = 'AirbnbDB';

-- Foreign-key count
SELECT COUNT(*) FROM information_schema.referential_constraints
WHERE constraint_schema = 'AirbnbDB';

-- Row count per table
SELECT table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'AirbnbDB'
ORDER BY table_name;
```

The first statement should return `21`, the second a value greater than `15`, and the third a result set in which every row count is at least `20`.

8. Re-Execution and Idempotency

The script is **not** idempotent. It is designed to be run on a freshly created database. To re-run the entire file, drop the existing database first:

```
<code class="language-sql">DROP DATABASE IF EXISTS AirbnbDB;
```

Then re-execute the script from the top.

9. Extension Points

The script is a coursework artefact, but the schema is sufficiently clean to support future work. The following extensions are realistic and would not require structural changes:

- Adding an index on `Booking(GuestID)` or `Accommodation(HostID)` to accelerate joins.
- Introducing a `Currency` table and converting the `Pricing` and `TotalPrice` columns to foreign keys.
- Adding a stored procedure that books an accommodation atomically, creating a `Booking`, a `Payment`, and a `Reservation` in a single transaction.
- Adding audit columns (`CreatedAt`, `UpdatedAt`) to each table to support change tracking.
- Building a thin REST layer on top of the three test-case queries for use in a reporting dashboard.

10. Summary

The SQL file is a single, self-contained script that builds the entire AirbnbDB data mart in five logical steps. Foreign-key constraints are enforced both at `CREATE TABLE` time and through targeted `ALTER TABLE` statements. Dummy data is provided for every table in sufficient volume to support realistic reporting queries. The three test cases described in the test-case document exercise the schema through multi-table joins and confirm that the data mart is internally consistent.

Database Metadata

Course: DLBDSPBDM01 – Build a Data Mart in SQL; **Programme:** B.Sc. Computer Science; **Author:** Zubair Ahmed Khan; **Matriculation Number:** 92127983; **Tutor:** Prof. Dr. Anna Androvitsanea; **Submission Date:** 2026-07-29

---

1. Identification

Attribute	Value
Database Name	**AirbnbDB**
Purpose	Coursework data mart modelling an Airbnb-style short-term rental platform, supporting users, guests, hosts, accommodations, bookings, reservations, payments, reviews, complaints, notifications, amenities, locations, and listings.
Version	1.0

2. Technical Environment

Attribute	Value
Database Management System	**MySQL Server 8.0**
Client Tool	**MySQL Workbench 8.0**
SQL Language	**SQL** (DDL and DML)
Storage Engine	InnoDB (default)
Character Set	utf8mb4 (default)
Server Platform	Cross-platform (Windows, macOS, Linux)

3. Schema Statistics

Attribute	Value
Number of Tables	**21**
Number of Entities	**21** (one-to-one with tables)
Number of Relationships	One recursive (`Employee–Manager`); one ternary (`Booking–Payment–Reservation`); one many-to-many (`Place–Amenity` through `AmenityListing`); plus a network of one-to-many and one-to-one links
Primary Keys	**21** — one per table, integer, named `<TableName>ID`
Foreign Keys	Implemented across dependent tables; reinforced by `ALTER TABLE` for late-bound references
Minimum Records per Table	**20**
Total SQL Statements	**70+** (`CREATE`, `ALTER`, `INSERT`, `SELECT`, `SHOW`)
Estimated Storage Footprint	Less than 1 MB (small data volume; exact size depends on server configuration and `BLOB` allocation)

4. Normalisation and Integrity

Attribute	Value
Normalisation	**1NF, 2NF, 3NF** (First, Second, and Third Normal Form)
Primary Key Strategy	Surrogate integer key per table
Foreign Key Strategy	Declarative `FOREIGN KEY` constraints; orphan rows rejected
Atomicity	All attributes hold single-valued data; no repeating groups
Dependency Hygiene	Non-key attributes depend on the whole primary key, not on other non-key attributes
Referential Integrity	**Enforced** through foreign-key constraints
Domain Integrity	Enforced through data types (`DECIMAL(10,2)`, `DATE`, `DATETIME`, `VARCHAR`, `BLOB`)

5. Testing

Attribute	Value
Number of Test Cases	**3**
Test Case 1	Retrieval of complete booking information (multi-table join over `Booking`, `Guest`, `User`, `Accommodation`)
Test Case 2	Calculation of host accommodation statistics and income (aggregation over `Accommodation`, `Booking`, `Host`, `User`)
Test Case 3	Retrieval of complete accommodation information (join over `Accommodation`, `Host`, `User`)
Verification Method	Multi-table `JOIN` statements returning non-empty result sets; `SELECT COUNT(*)` per table returning ≥ 20

6. Database Features

Feature	Description
Referential Integrity	Foreign-key constraints reject inserts and updates that would break parent–child links
Dummy Data	At least twenty realistic rows per table, sufficient for non-trivial joins and aggregations
Recursive Relationship	`Employee` references itself through `ManagerID` to model an organisational hierarchy
Ternary Relationship	`Booking`, `Payment`, and `Reservation` form a triple association representing the full reservation lifecycle
Many-to-Many Link	`AmenityListing` resolves the many-to-many between `Place` and `Amenity`
Schema Evolution	`ALTER TABLE` statements add new columns and foreign keys without data loss
Consistency	All non-key attributes depend on the primary key only (3NF); update anomalies are prevented
Scalability	The schema accepts new users, hosts, accommodations, and bookings without structural change
Portability	Pure SQL, no proprietary extensions; runs on any MySQL 8.0 server

7. Repository and Distribution

Attribute	Value
GitHub Repository	https://github.com/zbr-khn/SQL
Submission Folder	`DLBDSPBDM01_Final_Submission/`
SQL File Location	`DLBDSPBDM01_Final_Submission/SQL/airbnb.sql`
Documentation	Abstract, Installation Manual, SQL Documentation, Database Metadata, Submission Checklist
Total Deliverables	Combined PDF portfolio + ZIP folder + individual PDFs

8. Inventory of Tables

#	Table	Purpose
1	`User`	Platform participants (guests and hosts)
2	`Guest`	Specialisation of `User` for booking stays
3	`Host`	Specialisation of `User` for listing accommodations
4	`Accommodation`	Property listed by a host
5	`Booking`	Reservation of an accommodation by a guest
6	`Payment`	Financial settlement of a booking
7	`Reservation`	Confirmed reservation linking booking and payment
8	`Review`	Guest-to-host or host-to-guest review
9	`Complaint`	Issue raised by a guest or host
10	`Notification`	System message directed to a user
11	`Location`	Geographic address (country, region, city, street, post code)
12	`HouseRules`	Set of rules associated with a place
13	`Place`	Physical description of a property (rooms, size, max guests)
14	`Amenity`	Catalogue of amenities (WiFi, AC, pool, etc.)
15	`AmenityListing`	Many-to-many link between `Place` and `Amenity`
16	`Media`	Photos and other media assets
17	`Listing`	Public listing of a place with cover photo and description
18	`Employee`	Back-office staff with a self-referencing manager relationship
19	`Language`	Catalogue of spoken languages

#	Table	Purpose
20	`UserLanguage`	Many-to-many link between `User` and `Language`
21	`Income`	Per-host income record

9. Maintenance Notes

• The script is **not idempotent**; drop the database before re-running.

• The `ALTER TABLE` block in Section D is essential; without it, the `PlaceID` foreign key in `Accommodation` is not created.

• The `PaymentDate` column is added after the `Payment` inserts through a separate `ALTER TABLE`, in order to preserve the linear execution order of the script.

• For very large datasets, indexes on `Booking.GuestID`, `Booking.AccommodationID`, `Accommodation.HostID`, `Reservation.BookingID`, and `Reservation.PaymentID` would accelerate the test-case joins.

Final Submission Checklist

Course: DLBDSPBDM01 – Build a Data Mart in SQL; **Programme:** B.Sc. Computer Science; **Author:** Zubair Ahmed Khan; **Matriculation Number:** 92127983; **Tutor:** Prof. Dr. Anna Androvitsanea; **Submission Date:** 2026-07-29

---

1. Purpose

This checklist verifies that every artefact required by the IU International University of Applied Sciences module **DLBDSPBDM01 — Build a Data Mart in SQL** is present in the final submission before upload. Every box must be ticked.

2. Phase 1 — Requirements, ER Diagram, and Data Dictionary

✔	Deliverable	File / Location
■	Phase 1 PDF	`01_Phase1_Requirements_ER_DataDictionary.pdf`
■	Requirements Specification	Included in Phase 1 PDF
■	Requirements Analysis (Roles, Actions, Data, Functions)	Included in Phase 1 PDF
■	ER Diagram (21 entities)	Included in Phase 1 PDF
■	Recursive Relationship (Employee–Manager)	Included in ER Diagram
■	Triple Relationship (Booking–Payment–Reservation)	Included in ER Diagram
■	Data Dictionary	Included in Phase 1 PDF
■	Phase 1 Summary	Included in Phase 1 PDF
■	References	Included in Phase 1 PDF

3. Phase 2 — Database Implementation

✔	Deliverable	File / Location
■	Phase 2 PDF	`02_Phase2_Database_Implementation.pdf`
■	Complete MySQL Database (21 tables)	`SQL/airbnb.sql`
■	SQL Creation Script	`SQL/airbnb.sql` (Section A, B, C, D)
■	SQL Insert Statements	`SQL/airbnb.sql` (Section B, E)
■	Primary Keys	Declared in every `CREATE TABLE`
■	Foreign Keys	Declared in `CREATE TABLE` and reinforced by `ALTER TABLE`
■	At least 20 records in every table	Verified by `SELECT COUNT(*)` per table
■	Screenshots of `CREATE TABLE` execution	Included in Phase 2 PDF
■	Screenshots of table contents	Included in Phase 2 PDF
■	Explanation for every entity	Included in Phase 2 PDF
■	ER Diagram Slide	Included in Presentation
■	Introduction Slide	Included in Presentation
■	Humanised Technical Overview	Included in Presentation
■	Test Case 1 — Retrieval of complete booking information	Included in Phase 2 PDF
■	Test Case 2 — Host statistics and income calculation	Included in Phase 2 PDF
■	Test Case 3 — Retrieval of complete accommodation info	Included in Phase 2 PDF
■	SQL query for each test case	Included in Phase 2 PDF
■	Explanation and result screenshot for each test case	Included in Phase 2 PDF

4. Phase 3 — Finalisation

✔	Deliverable	File / Location
■	Phase 3 Abstract	`03_Phase3_Abstract.pdf`
■	Installation Manual	`Installation_Manual.pdf`
■	SQL Documentation	`SQL_Documentation.pdf`
■	Database Metadata Page	`Database_Metadata.pdf`
■	Final Submission Checklist	`Final_Submission_Checklist.pdf` (this document)

5. Presentation

✔	Deliverable	File / Location
■	Presentation PDF	`Presentation.pdf`
■	At least 20 slides	Verified
■	One slide per entity	21 entity slides
■	SQL statement per entity	Verified
■	Screenshots of SQL execution	Verified
■	Screenshots of table contents	Verified

6. SQL Artefacts

✔	Deliverable	File / Location
■	SQL file `airbnb.sql`	`SQL/airbnb.sql`
■	Database `AirbnbDB` created	Verified by `SHOW DATABASES;`
■	21 tables created	Verified by `SHOW TABLES;`
■	Primary keys declared	Verified
■	Foreign keys declared	Verified
■	Dummy data inserted (≥ 20 rows per table)	Verified by `SELECT COUNT(*)` per table
■	Referential integrity enforced	Verified by failed orphan inserts
■	Script executes without errors	Verified
■	Test Case 1 returns rows	Verified
■	Test Case 2 returns rows	Verified
■	Test Case 3 returns rows	Verified

7. Combined PDF and Archive

✔	Deliverable	File / Location
■	Combined Portfolio PDF	`20250628_Zubair_Khan_92127983_DLBDSPBDM01_P3.pdf`
■	ZIP Folder	`Khan-Zubair_92127983_DLBDSPBDM01_Data_Mart_Submission_all.zip`
■	GitHub Repository link	`GitHub_Link.txt`
■	Public GitHub Repository accessible	Verified
■	README file present	`README.txt`

8. Database Metadata (One-Page Summary)

✔	Item	Confirmed
■	Database Name (AirbnbDB)	■
■	Purpose	■
■	Database Management System (MySQL)	■
■	SQL Language	■
■	Number of Tables (21)	■
■	Number of Entities (21)	■
■	Number of Relationships (including recursive and ternary)	■
■	Primary Keys (21)	■
■	Foreign Keys	■
■	Minimum Records per Table (20)	■
■	Normalisation (1NF, 2NF, 3NF)	■
■	Test Cases (3)	■
■	GitHub Repository	■
■	Total SQL Statements (70+)	■
■	Database Features	■
■	Referential Integrity	■
■	Dummy Data	■
■	Scalability	■

9. Academic Integrity

✔	Item	Confirmed
■	All work is original	■
■	No third-party content used without attribution	■
■	Sources properly cited in references	■
■	No AI-generated text presented as own without editing	■
■	Plagiarism declaration understood	■

10. Final Sign-Off

Item	Value
Author	Zubair Khan
Matriculation Number	92127983
Course Code	DLBDSPBDM01
Submission Date	2026-07-29
All checkboxes above ticked	■ Yes
Ready for upload	■ Yes

Signature: _____ **Date:** _____